



第5章

网络优化与正则化

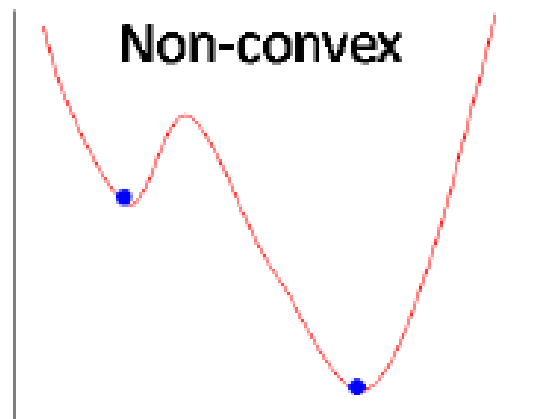
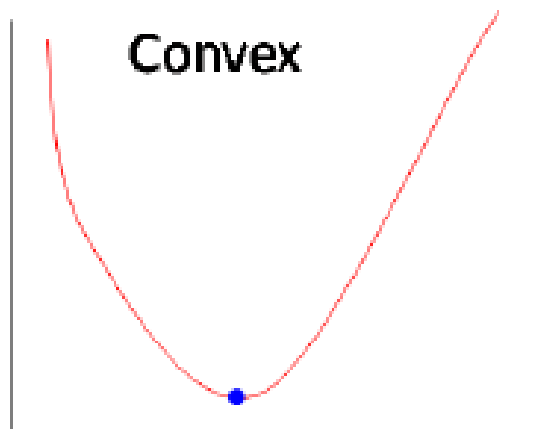
机器学习的矛与盾

优化
经验风险最小

正则化
降低模型复杂度



机器学习的矛与盾

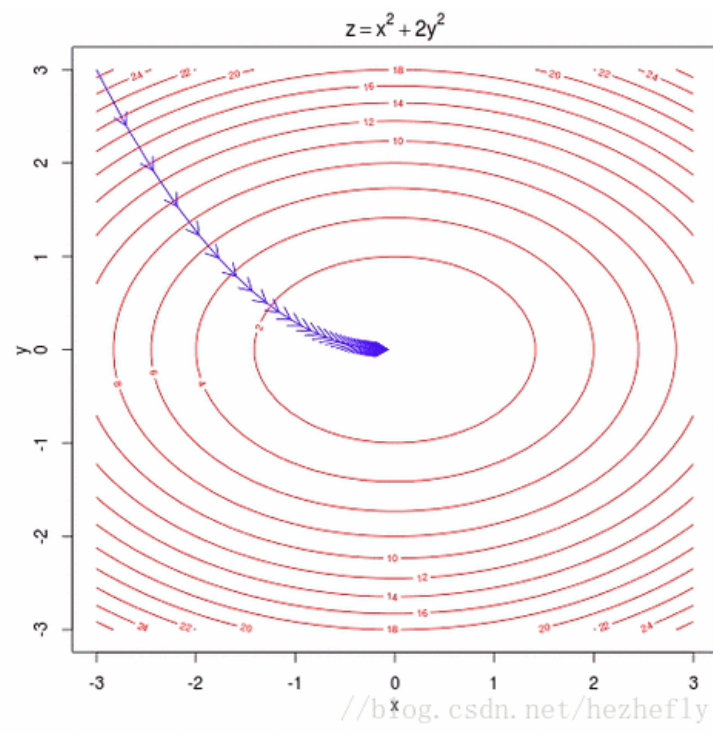
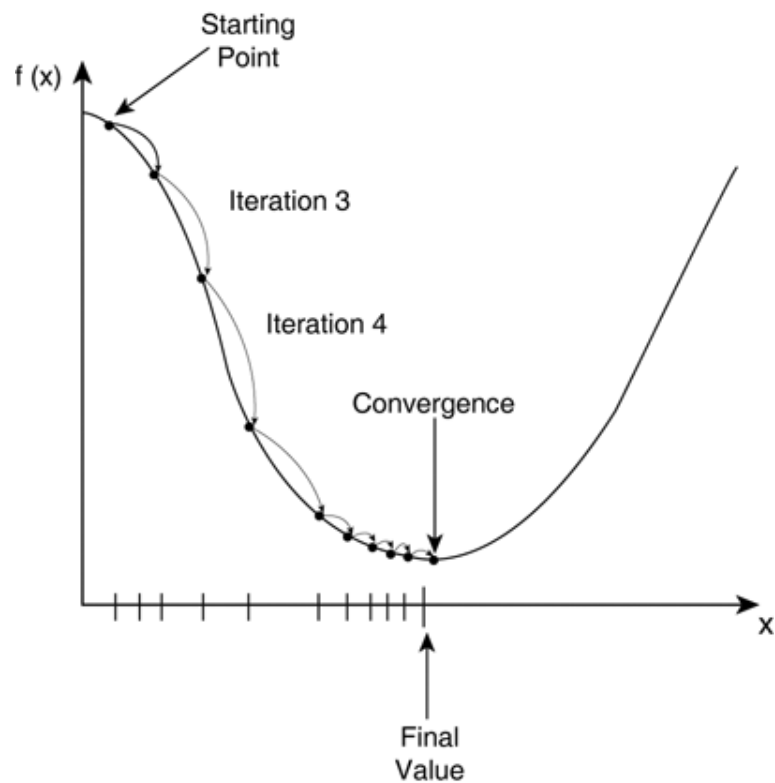


$$\min_{\mathbf{x}} f(\mathbf{x})$$

$$\mathcal{R}(f) = \mathbb{E}_{(\mathbf{x}, y) \sim p(\mathbf{x}, y)} [\mathcal{L}(f(\mathbf{x}), y)],$$

$$\mathcal{R}_D^{emp}(\theta) = \frac{1}{N} \sum_{n=1}^N \mathcal{L}(y^{(n)}, f(x^{(n)}, \theta))$$

机器学习的矛与盾



$$\begin{aligned}\theta_{t+1} &= \theta_t - \alpha \frac{\partial \mathcal{R}(\theta)}{\partial \theta_t} \\ &= \theta_t - \alpha \frac{1}{N} \sum_{i=1}^N \frac{\partial \mathcal{L}(\theta_t; x^{(i)}, y^{(i)})}{\partial \theta}.\end{aligned}$$

虽然神经网络具有非常强的表达能力,但是当应用神经网络模型到机器学习时依然存在一些难点问题. 主要分为两大类:

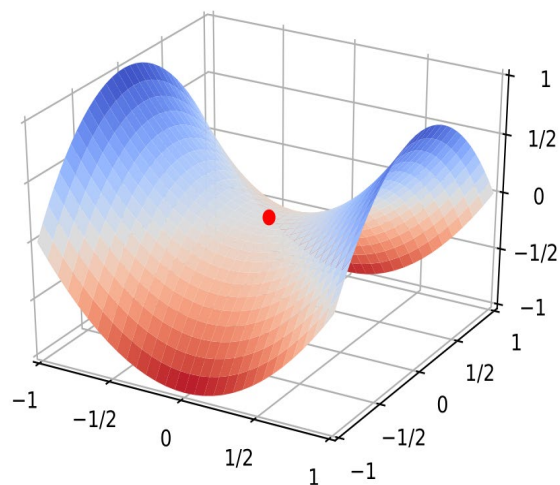
(1) **优化问题**: 深度神经网络的优化十分困难. 首先,神经网络的损失函数是一个非凸函数, 找到全局最优解通常比较困难. 其次,深度神经网络的参数通常非常多,训练数据也比较大,因此也无法使用计算代价很高的二阶优化方法,而一阶优化方法的训练效率通常比较低. 此外,深度神经网络存在梯度消失或爆炸问题,导致基于梯度的优化方法经常失效.

(2) **泛化问题**: 由于深度神经网络的复杂度比较高,并且拟合能力很强,很容易在训练集上产生过拟合. 因此在训练深度神经网络时,同时也需要通过一定的**正则化**方法来改进网络的泛化能力.

高维空间的非凸优化问题

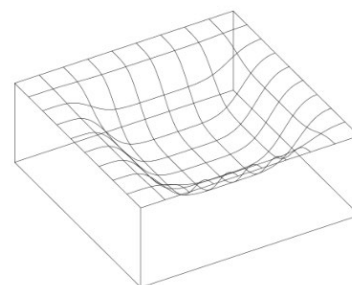
● 鞍点 (Saddle Point)

- 驻点 (Stationary Point) : 梯度为 0 的点。

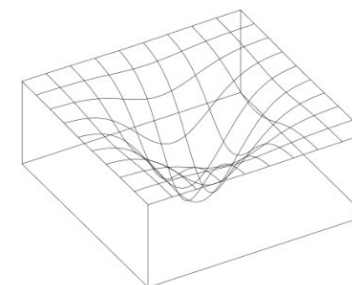


● 平坦最小值 (Flat Minima)

- 一个平坦最小值的邻域内, 所有点对应的训练损失都比较接近
- 大部分的局部最小解是等价的
- 局部最小解对应的训练损失都可能非常接近于全局最小解对应的训练损失



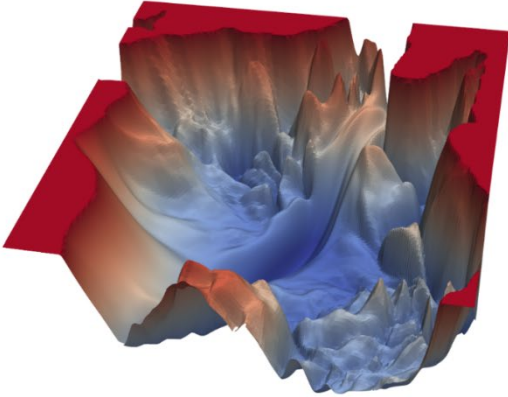
(a) 平坦最小值



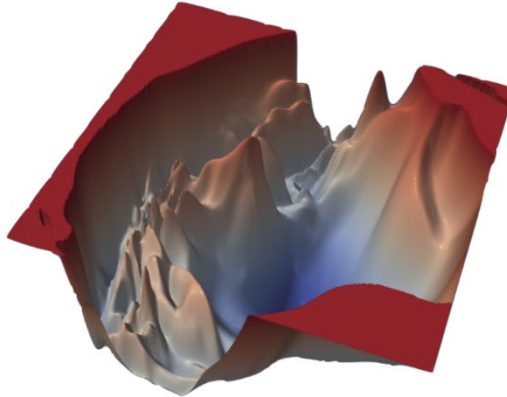
(b) 尖锐最小值

Loss Landscape

VGG-56

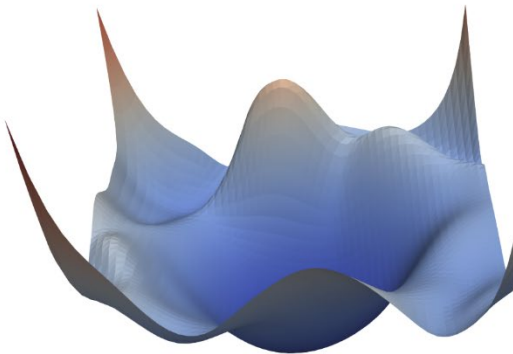


VGG-110

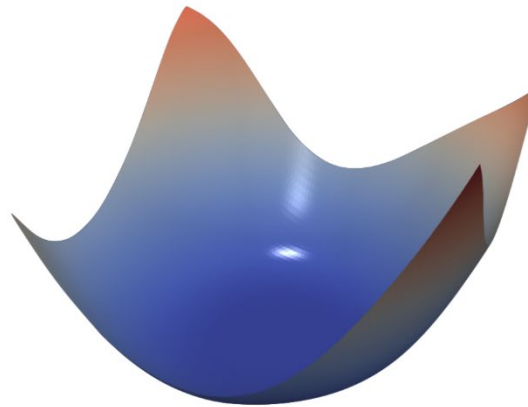


without skip connections

Resnet-56



Densenet-121



with skip connections

神经网络优化的改善方法

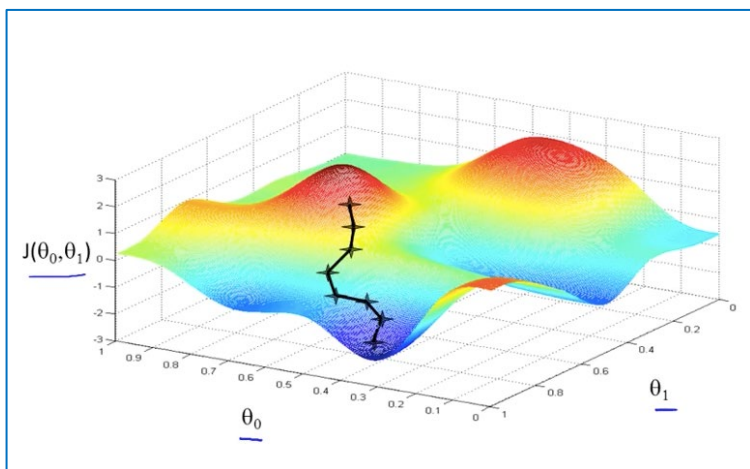
- 更有效的优化算法来提高优化方法的效率和稳定性
 - 动态学习率调整
 - 梯度估计修正
- 更好的参数初始化方法、数据预处理方法来提高优化效率
- 修改网络结构来得到更好的优化地形
 - 优化地形（ Optimization Landscape ）指在高维空间中损失函数的曲面形状
 - 好的优化地形通常比较平滑
 - 使用 ReLU 激活函数、残差连接、逐层归一化等
- 使用更好的超参数优化方法

改进优化算法



优化算法：随机梯度下降

- 批量梯度下降
- 随机梯度下降
- 小批量梯度下降



算法 2.1: 随机梯度下降法

输入: 训练集 $\mathcal{D} = \{(\mathbf{x}^{(n)}, y^{(n)})\}_{n=1}^N$, 验证集 $\mathcal{V}$, 学习率 α

```
1 随机初始化  $\theta$ ;  
2 repeat  
3   对训练集  $\mathcal{D}$  中的样本随机重排序;  
4   for  $n = 1 \cdots N$  do  
5     从训练集  $\mathcal{D}$  中选取样本  $(\mathbf{x}^{(n)}, y^{(n)})$ ;  
6     // 更新参数  
        $\theta \leftarrow \theta - \alpha \frac{\partial \mathcal{L}(\theta; x^{(n)}, y^{(n)})}{\partial \theta};$   
7   end  
8 until 模型  $f(\mathbf{x}, \theta)$  在验证集  $\mathcal{V}$  上的错误率不再下降;  
   输出:  $\theta$ 
```

小批量梯度下降

- 选取K个训练样本，计算偏导数

$$\mathbf{g}_t(\theta) = \frac{1}{K} \sum_{(\mathbf{x}^{(k)}, \mathbf{y}^{(k)}) \in \mathcal{I}_t} \frac{\partial \mathcal{L}(\mathbf{y}^{(k)}, f(\mathbf{x}^{(k)}, \theta))}{\partial \theta}$$

- 定义梯度

$$\mathbf{g}_t \triangleq \mathbf{g}_t(\theta_{t-1})$$

- 更新参数

$$\theta_t \leftarrow \theta_{t-1} - \alpha \mathbf{g}_t$$

其中 $\alpha > 0$ 为学习率

几个关键因素：

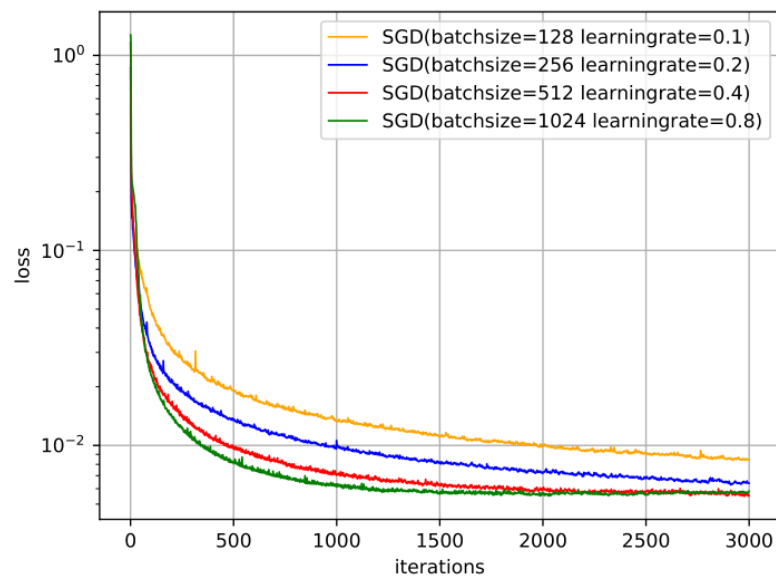
- 小批量样本数量
- 梯度
- 学习率

批量大小的影响

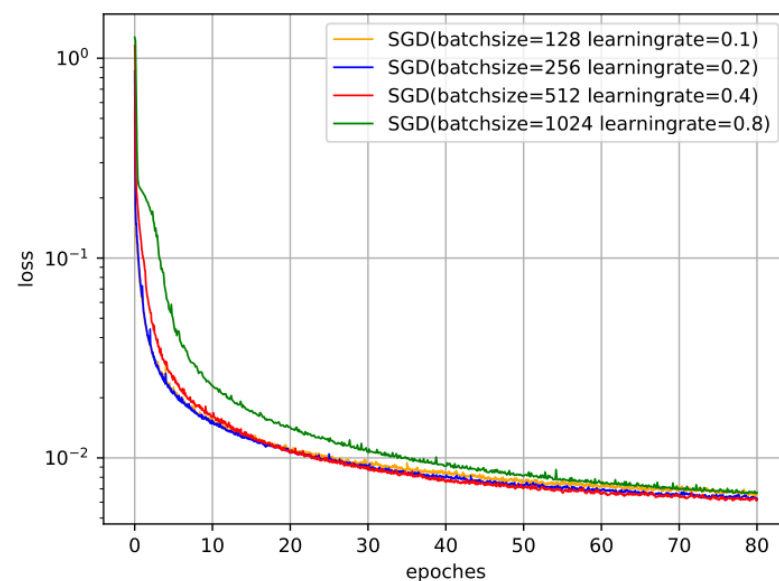
- 批量大小不影响随机梯度的期望，但是会影响随机梯度的方差

- 批量越大，随机梯度的方差越小，引入的噪声也越小，训练也越稳定，因此可以设置较大的学习率
- 而批量较小时，需要设置较小的学习率，否则模型会不收敛

$$1 \text{ 回合 (Epoch)} = \left(\frac{\text{训练样本的数量 } N}{\text{批量大小 } K} \right) \times \text{迭代 (Iteration)} .$$



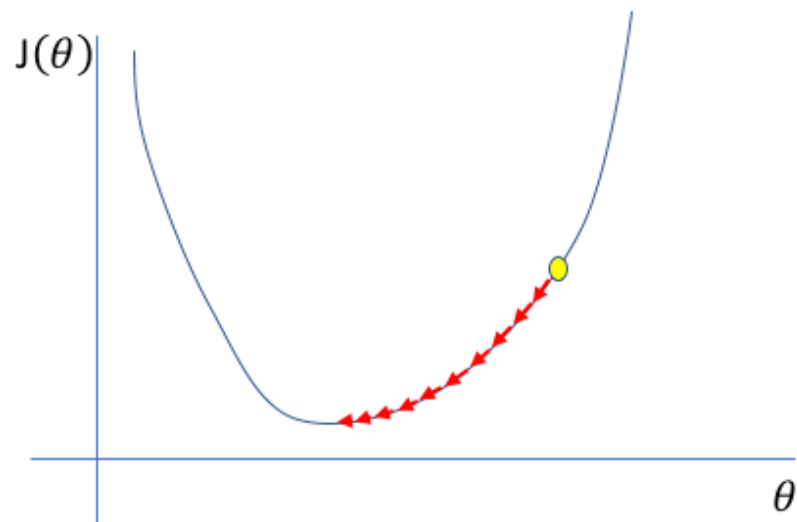
(a) 按 Iteration 的损失变化



(b) 按 Epoch 的损失变化

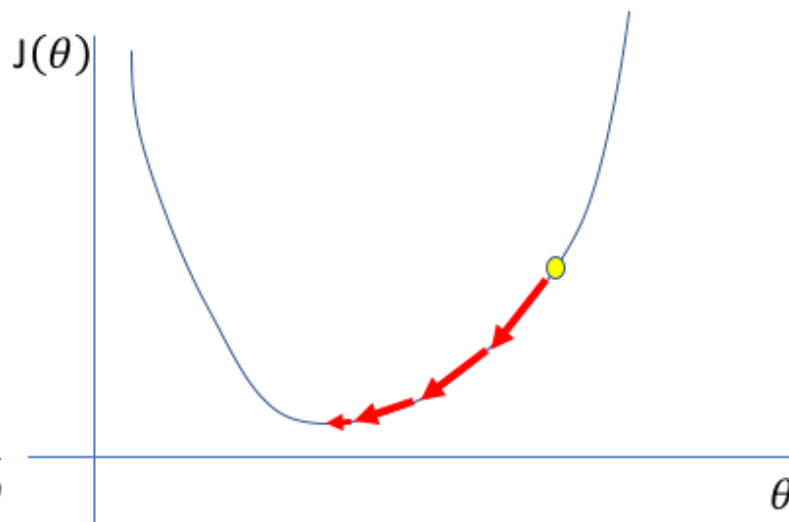
学习率的影响

Too low



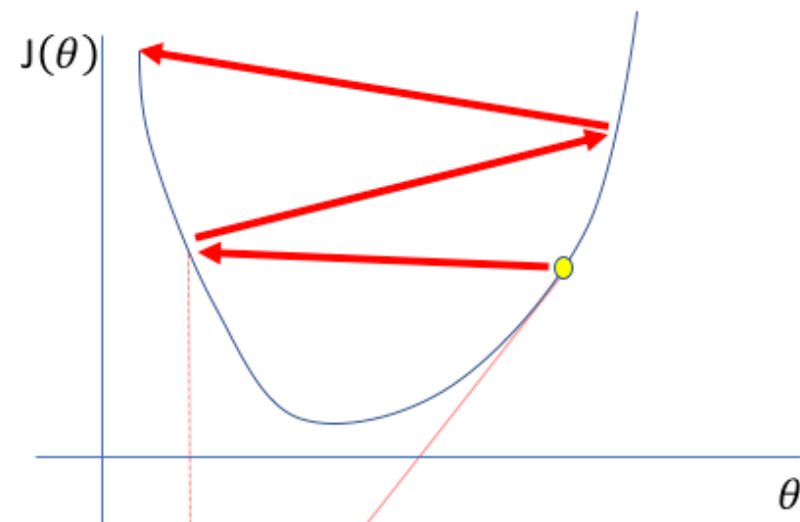
A small learning rate requires many updates before reaching the minimum point

Just right



The optimal learning rate swiftly reaches the minimum point

Too high



Too large of a learning rate causes drastic updates which lead to divergent behaviors

学习率衰减/退火 (Learning Rate Decay / Annealing)

逆时衰减 (Inverse Time Decay) :

$$\alpha_t = \alpha_0 \frac{1}{1 + \beta \times t},$$

其中 β 为衰减率.

指数衰减 (Exponential Decay) :

$$\alpha_t = \alpha_0 \beta^t,$$

其中 $\beta < 1$ 为衰减率.

自然指数衰减 (Natural Exponential Decay) :

$$\alpha_t = \alpha_0 \exp(-\beta \times t),$$

其中 β 为衰减率.

余弦衰减 (Cosine Decay) :

$$\alpha_t = \frac{1}{2} \alpha_0 \left(1 + \cos\left(\frac{t\pi}{T}\right) \right),$$

其中 T 为总的迭代次数.

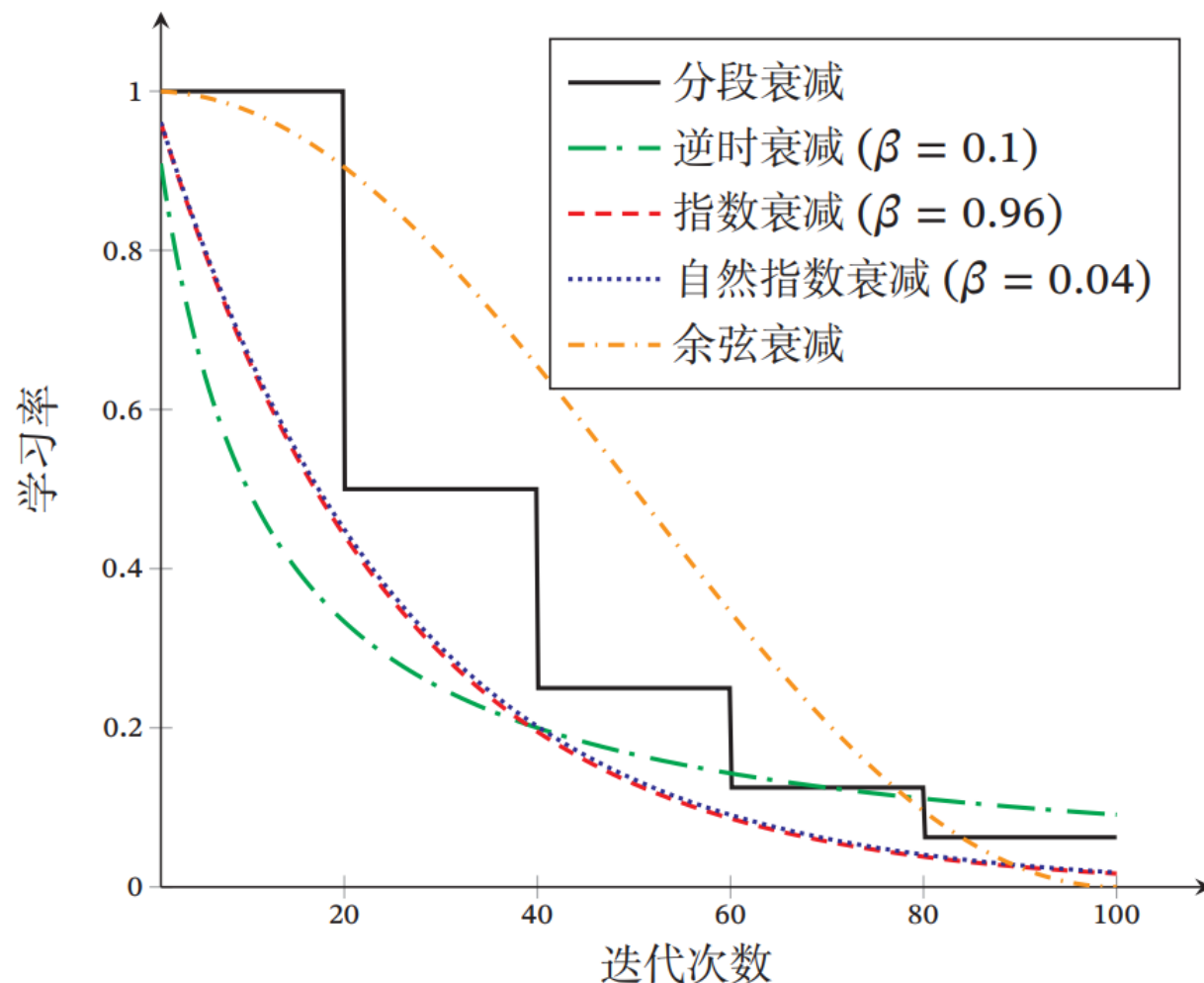


图 7.4 不同学习率衰减方法的比较

学习率预热 (Learning Rate Warmup)

● 问题

- 在小批量梯度下降法中，当批量大小的设置比较大时，通常需要比较大的学习率。
- 但在刚开始训练时，由于参数是随机初始化的，梯度往往也比较大，再加上比较大的初始学习率，会使得训练不稳定。

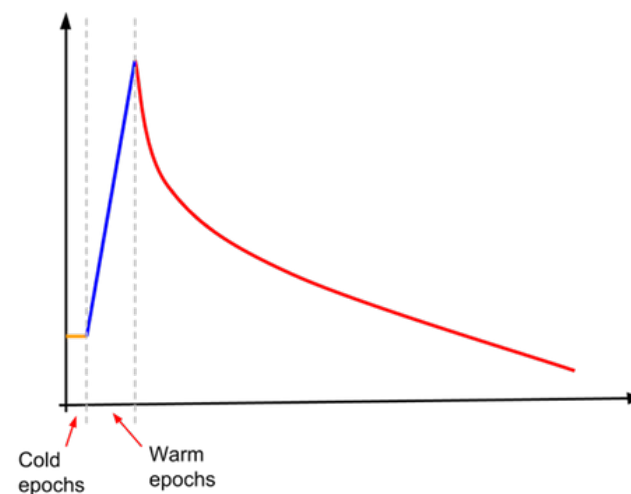
● 解决方案

为了提高训练稳定性，我们可以在最初几轮迭代时，采用比较小的学习率，等梯度下降到一定程度后再恢复到初始的学习率，这种方法称为学习率预热 (Learning Rate Warmup)。

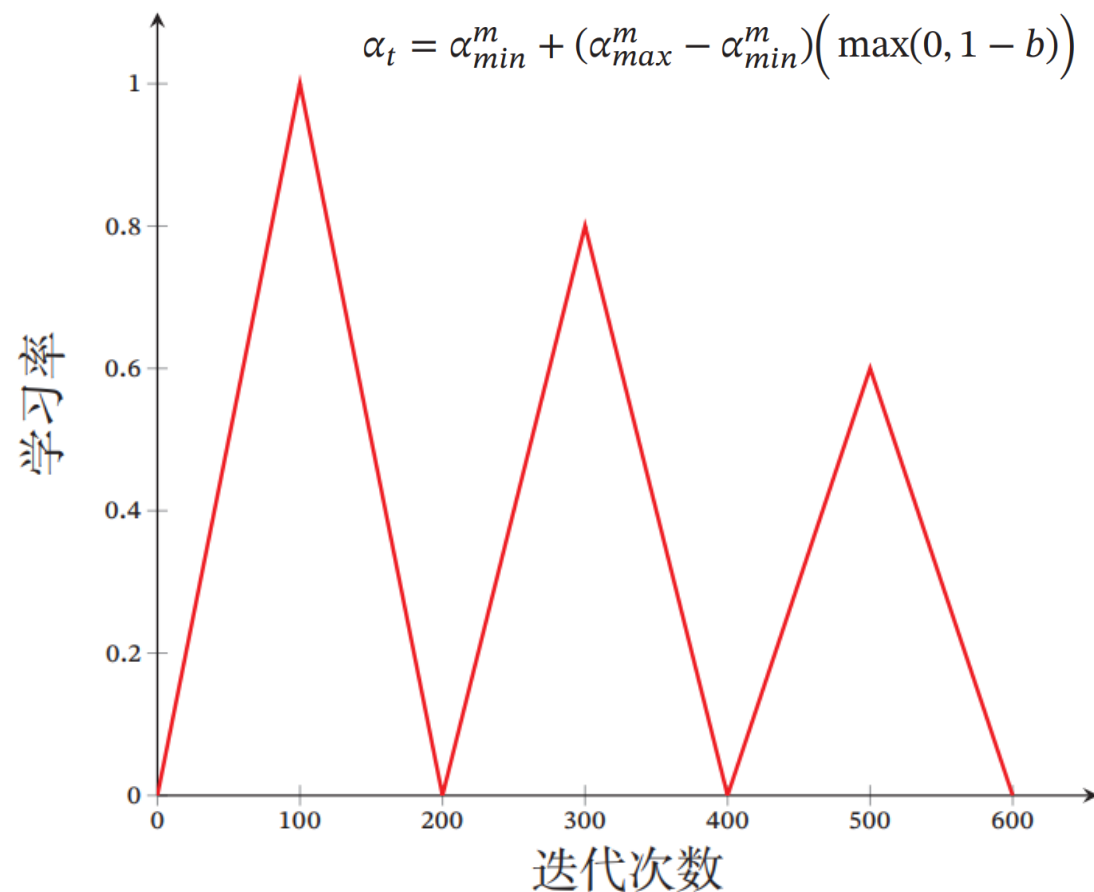
一个常用的学习率预热方法是逐渐预热 (Gradual Warmup) [Goyal et al., 2017]。假设预热的迭代次数为 T' ，初始学习率为 α_0 ，在预热过程中，每次更新的学习率为

$$\alpha'_t = \frac{t}{T'} \alpha_0, \quad 1 \leq t \leq T'. \quad (7.9)$$

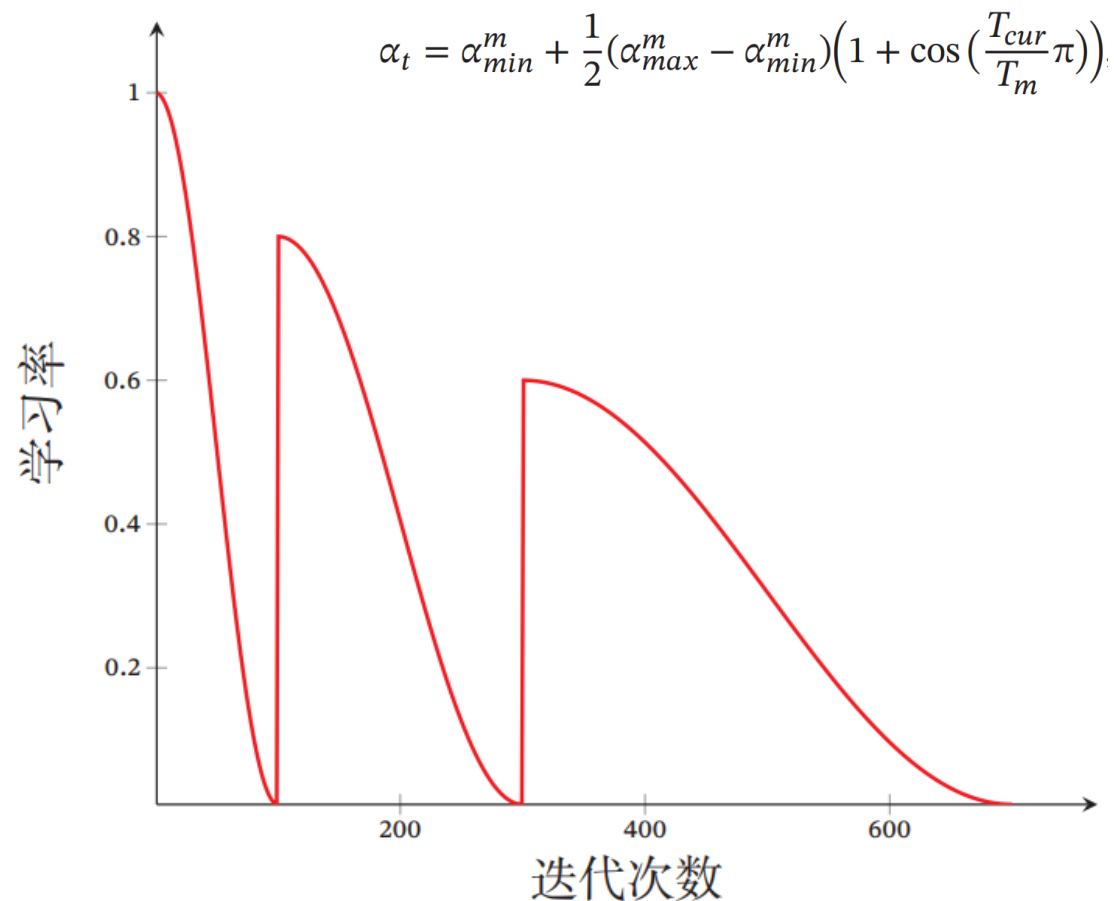
当预热过程结束，再选择一种学习率衰减方法来逐渐降低学习率。



周期性学习率调整 (Cyclical Learning Rates)



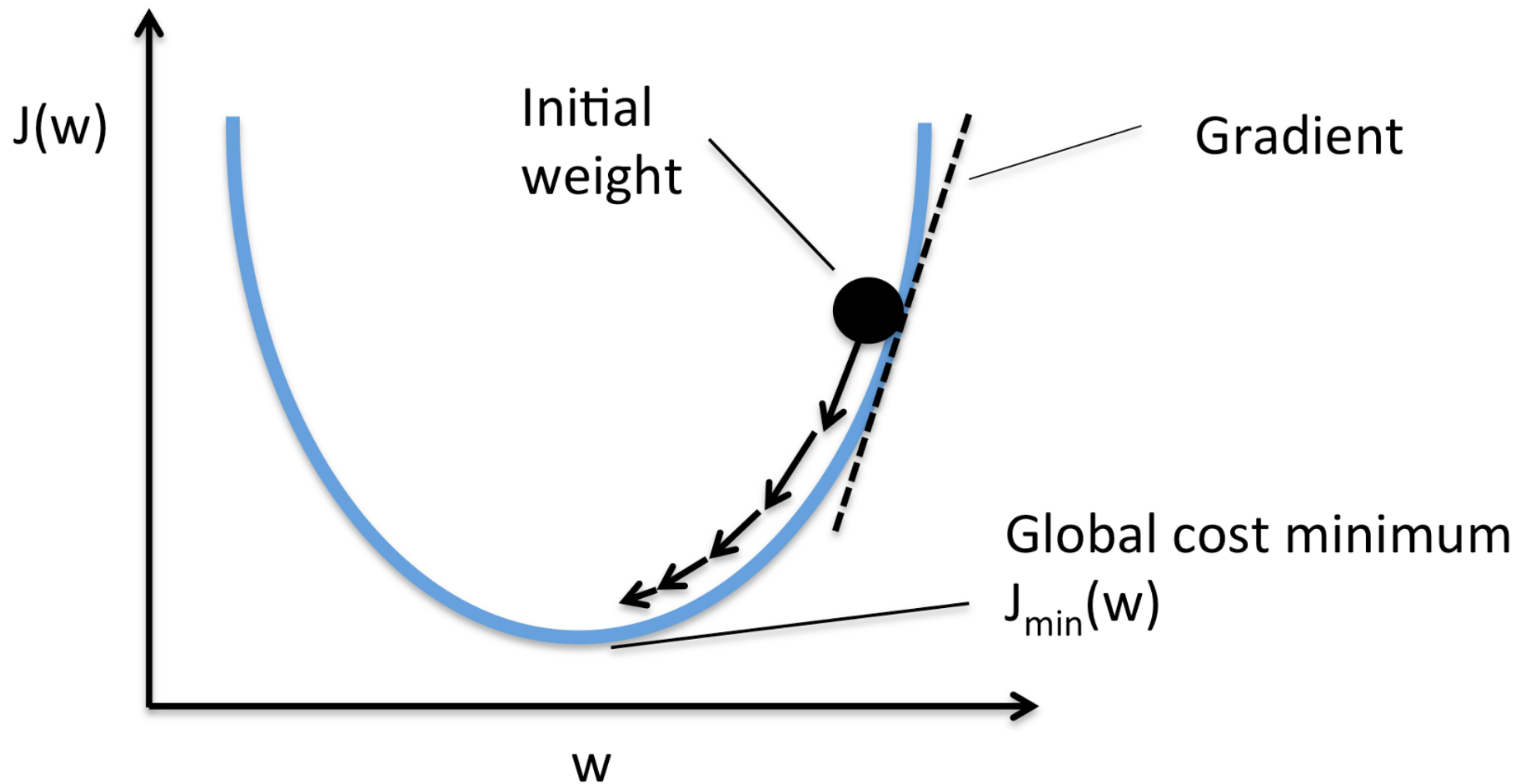
(a) 三角循环学习率



(b) 带热重启的余弦衰减

梯度估计修正

$$w_{t+1} = w_t - \eta \hat{\nabla}_w L(w_t)$$



AdaGrad算法 (Adaptive Gradient Algorithm)

AdaGrad 算法 (Adaptive Gradient Algorithm) [Duchi et al., 2011] 是借鉴 ℓ_2 正则化的思想, 每次迭代时自适应地调整每个参数的学习率. 在第 t 次迭代时, 先计算每个参数梯度平方的累计值

$$G_t = \sum_{\tau=1}^t \mathbf{g}_{\tau} \odot \mathbf{g}_{\tau},$$

其中 $\odot$ 为按元素乘积, $\mathbf{g}_{\tau} \in \mathbb{R}^{|\theta|}$ 是第 τ 次迭代时的梯度.

AdaGrad 算法的参数更新差值为

$$\Delta\theta_t = -\frac{\alpha}{\sqrt{G_t + \epsilon}} \odot \mathbf{g}_t,$$

其中 α 是初始的学习率, ϵ 是为了保持数值稳定性而设置的非常小的常数, 一般取值 e^{-7} 到 e^{-10} . 此外, 这里的开平方、除、加运算都是按元素进行的操作.

在 AdaGrad 算法中,

- 如果某个参数的偏导数累积比较大, 其学习率相对较小;
- 相反, 如果其偏导数累积较小, 其学习率相对较大.
- 但整体是随着迭代次数的增加, 学习率逐渐缩小.

AdaGrad 算法的缺点:

- 在经过一定次数的迭代依然没有找到最优点时, 由于这时的学习率已经非常小, 很难再继续找到最优点.

RMSprop算法

RMSprop 算法首先计算每次迭代梯度 $\mathbf{g}_t$ 平方的指数衰减移动平均,

$$\begin{aligned} G_t &= \beta G_{t-1} + (1 - \beta) \mathbf{g}_t \odot \mathbf{g}_t \\ &= (1 - \beta) \sum_{\tau=1}^t \beta^{t-\tau} \mathbf{g}_\tau \odot \mathbf{g}_\tau, \end{aligned}$$

其中 β 为衰减率,一般取值为 0.9.

RMSprop 算法的参数更新差值为

$$\Delta \theta_t = -\frac{\alpha}{\sqrt{G_t + \epsilon}} \odot \mathbf{g}_t,$$

其中 α 是初始的学习率,比如 0.001.

RMSProp 算法和 AdaGrad 算法的区别:

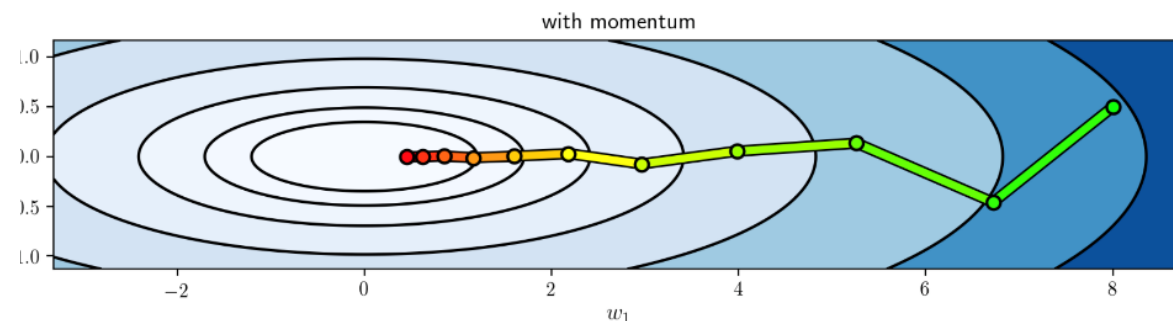
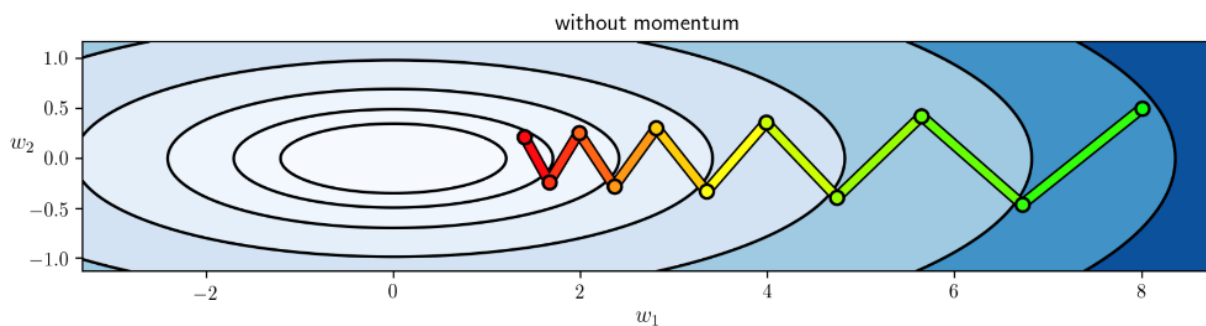
- G_t 的计算由累积方式变成了指数衰减移动平均.
- 在迭代过程中, 每个参数的学习率并不是呈衰减趋势, 既可以变小也可以变大.

动量法 (Momentum Method)

在第 t 次迭代时, 计算负梯度的“加权移动平均”作为参数的更新方向,

$$\Delta\theta_t = \rho\Delta\theta_{t-1} - \alpha\mathbf{g}_t = -\alpha \sum_{\tau=1}^t \rho^{t-\tau} \mathbf{g}_\tau, \quad (7.21)$$

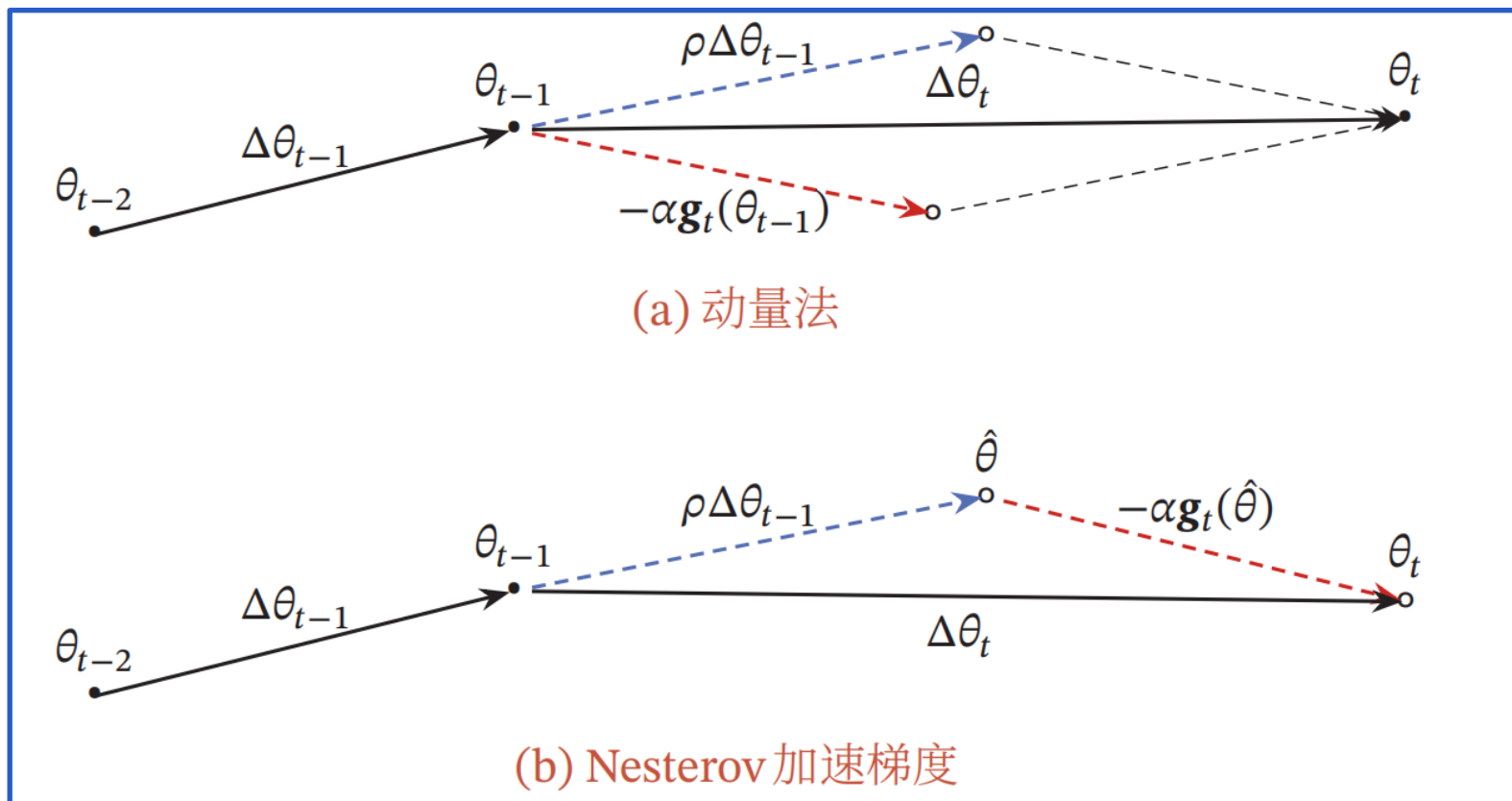
其中 ρ 为动量因子, 通常设为 0.9, α 为学习率.



Nesterov加速梯度 (Nesterov Accelerated Gradient, NAG)

$$\Delta\theta_t = \rho\Delta\theta_{t-1} - \alpha\mathbf{g}_t(\theta_{t-1} + \rho\Delta\theta_{t-1}), \quad (7.24)$$

其中 $\mathbf{g}_t(\theta_{t-1} + \rho\Delta\theta_{t-1})$ 表示损失函数在点 $\hat{\theta} = \theta_{t-1} + \rho\Delta\theta_{t-1}$ 上的偏导数.



Adam算法

Adam 算法一方面计算梯度平方 $\mathbf{g}_t^2$ 的指数加权平均 (和 RMSprop 算法类似), 另一方面计算梯度 $\mathbf{g}_t$ 的指数加权平均 (和动量法类似).

$$M_t = \beta_1 M_{t-1} + (1 - \beta_1) \mathbf{g}_t, \quad (7.25)$$

$$G_t = \beta_2 G_{t-1} + (1 - \beta_2) \mathbf{g}_t \odot \mathbf{g}_t, \quad (7.26)$$

其中 β_1 和 β_2 分别为两个移动平均的衰减率, 通常取值为 $\beta_1 = 0.9, \beta_2 = 0.99$.

$$\hat{M}_t = \frac{M_t}{1 - \beta_1^t}, \quad (7.27)$$

$$\hat{G}_t = \frac{G_t}{1 - \beta_2^t}. \quad (7.28)$$

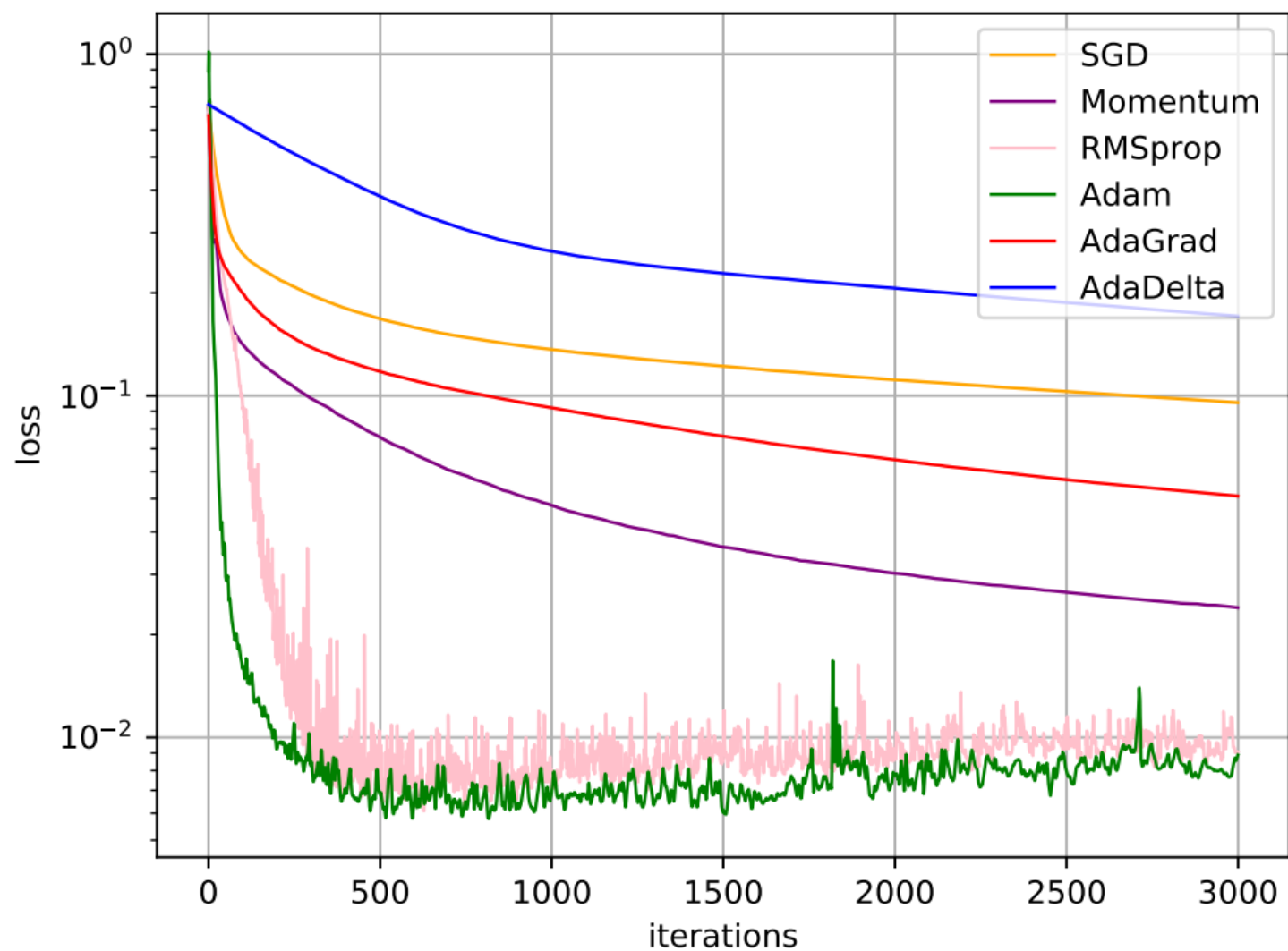
Adam 算法的参数更新差值为

$$\Delta \theta_t = -\frac{\alpha}{\sqrt{\hat{G}_t + \epsilon}} \hat{M}_t, \quad (7.29)$$

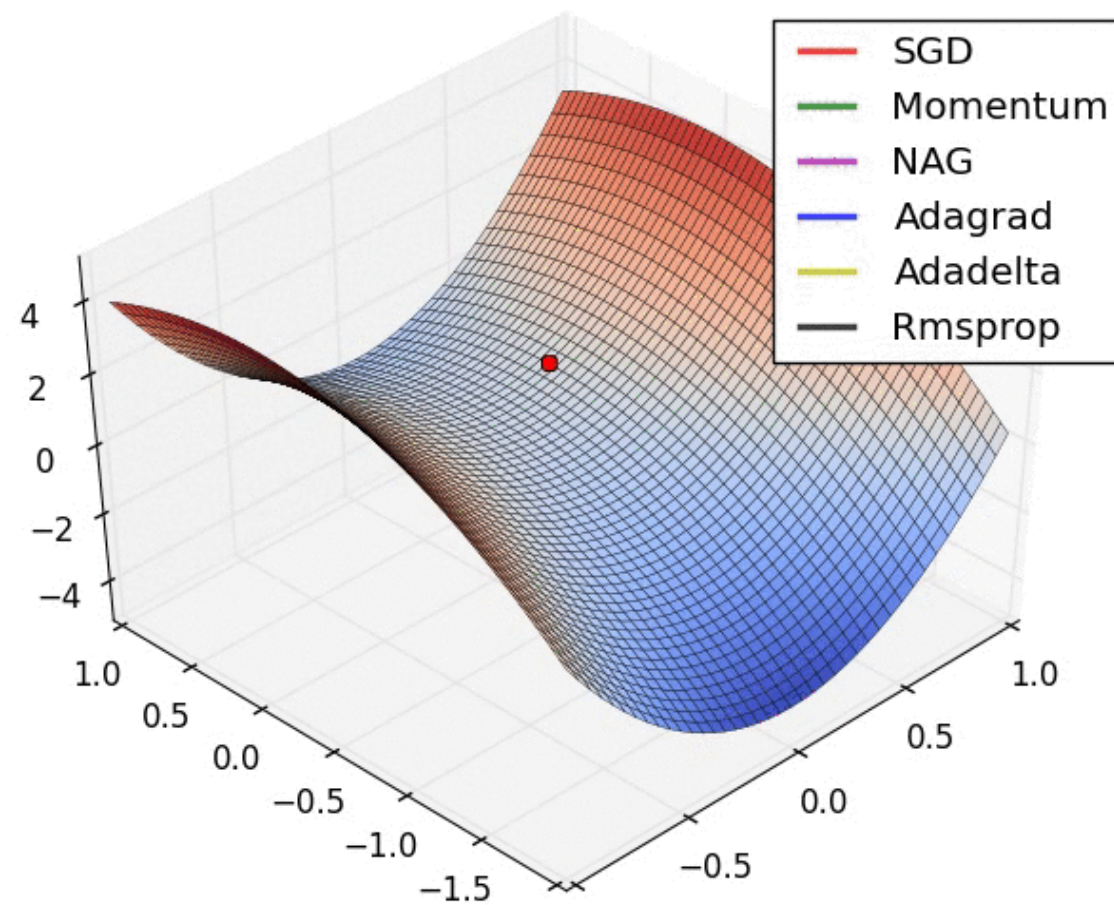
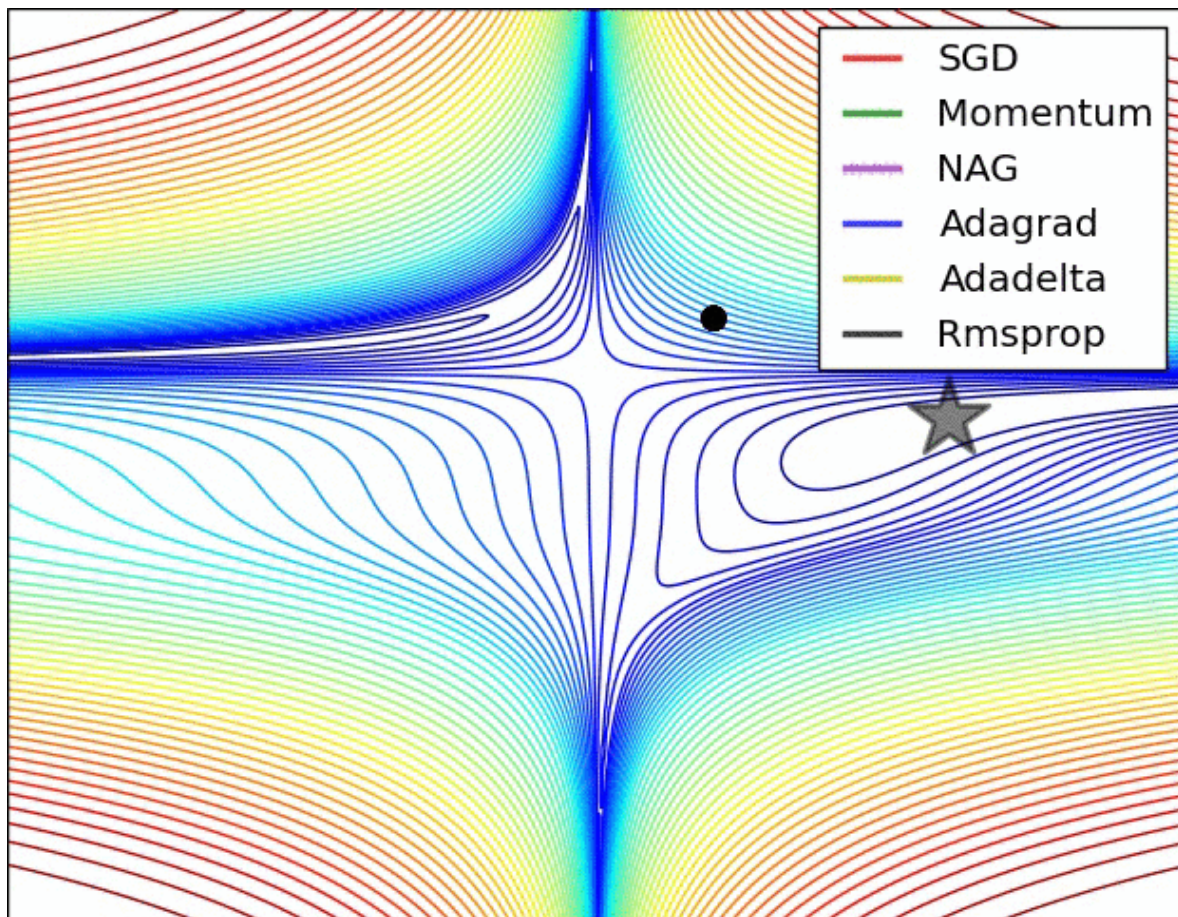
其中学习率 α 通常设为 0.001, 并且也可以进行衰减, 比如 $\alpha_t = \alpha_0 / \sqrt{t}$.

几种优化方法在 MNIST 数据集上收敛性的比较

- 学习率为0.001批量大小为128



优化方法



梯度截断 (Gradient Clipping)

- 在基于梯度下降的优化过程中，如果梯度突然增大，用大的梯度更新参数反而会导致其远离最优解
- 梯度截断是一种比较简单的启发式方法，把梯度的模限定在一个区间，当梯度的模小于或大于这个区间时就进行截断

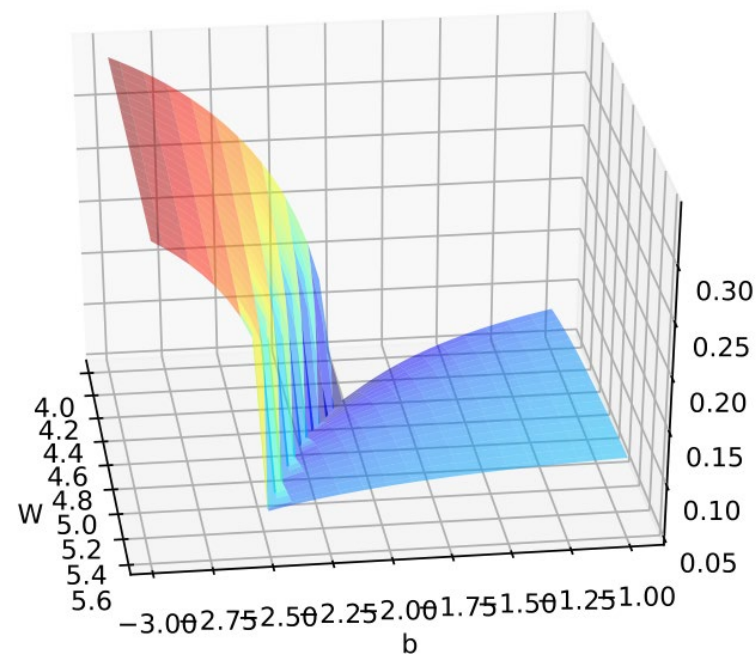
按值截断 在第 t 次迭代时，梯度为 $\mathbf{g}_t$ ，给定一个区间 $[a, b]$ ，如果一个参数的梯度小于 a 时，就将其设为 a ；如果大于 b 时，就将其设为 b 。

$$\mathbf{g}_t = \max(\min(\mathbf{g}_t, b), a). \quad (7.30)$$

按模截断 按模截断是将梯度的模截断到一个给定的截断阈值 b 。

如果 $\|\mathbf{g}_t\|^2 \leq b$ ，保持 $\mathbf{g}_t$ 不变。如果 $\|\mathbf{g}_t\|^2 > b$ ，令

$$\mathbf{g}_t = \frac{b}{\|\mathbf{g}_t\|} \mathbf{g}_t. \quad (7.31)$$



类别		优化算法
学习率调整	固定衰减学习率	分段常数衰减、逆时衰减、(自然)指数衰减、余弦衰减
	周期性学习率	循环学习率、SGDR
	自适应学习率	AdaGrad、RMSprop、AdaDelta
		$\Delta\theta_t = -\frac{\alpha_t}{\sqrt{G_t + \epsilon}}M_t,$ $G_t = \psi(\mathbf{g}_1, \cdots, \mathbf{g}_t),$ $M_t = \phi(\mathbf{g}_1, \cdots, \mathbf{g}_t),$
梯度估计修正		动量法、Nesterov 加速梯度、梯度截断
综合方法		Adam≈动量法+RMSprop

数据预处理和参数初始化



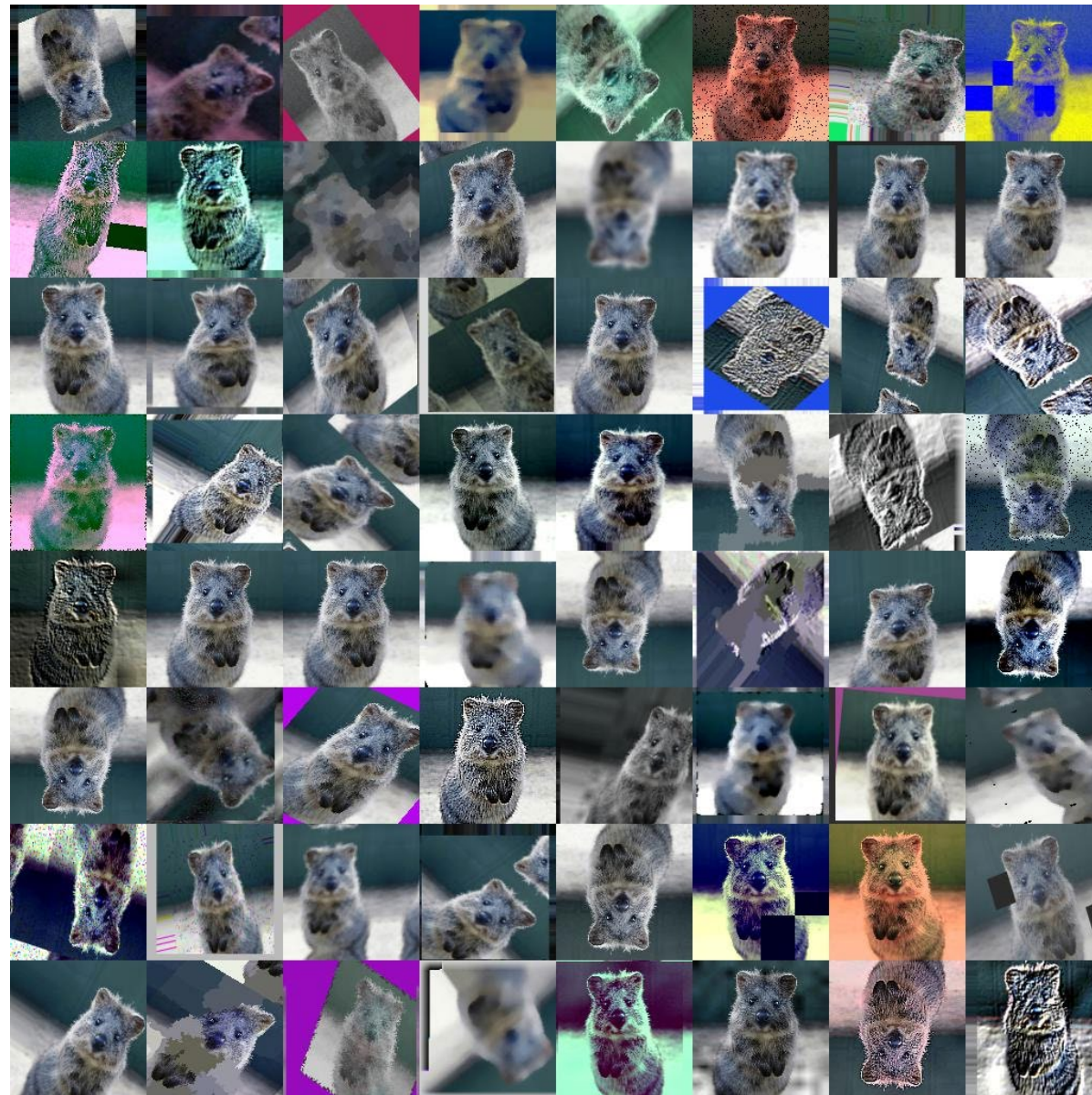
数据增强 (Data Augmentation)

- 增加数据的多样性

- 图像数据的增强方法:

图像数据的增强主要是通过算法对图像进行转变, 引入噪声等方法来增加数据的多样性. 增强的方法主要有几种:

- (1) 旋转 (Rotation): 将图像按顺时针或逆时针方向随机旋转一定角度.
- (2) 翻转 (Flip): 将图像沿水平或垂直方法随机翻转一定角度.
- (3) 缩放 (Zoom In/Out): 将图像放大或缩小一定比例.
- (4) 平移 (Shift): 将图像沿水平或垂直方法平移一定步长.
- (5) 加噪声 (Noise): 加入随机噪声.



标签平滑 (Label Smoothing)

- 在输出标签中添加噪声来避免模型过拟合。

- 一个样本x的标签一般用onehot向量表示

$$\mathbf{y} = [0, \dots, 0, 1, 0, \dots, 0]^T$$

- 引入一个噪声对标签进行平滑，即假设样本以 ϵ 的概率为其它类。平滑后的标签为

$$\tilde{\mathbf{y}} = \left[\frac{\epsilon}{K-1}, \dots, \frac{\epsilon}{K-1}, 1 - \epsilon, \frac{\epsilon}{K-1}, \dots, \frac{\epsilon}{K-1} \right]^T$$

- 知识蒸馏 (Knowledge Distillation)

- 教师模型、学生模型

参数初始化

- 预训练初始化 (Pre-trained Initialization)
 - 精调 (Fine-Tuning)
- 随机初始化 (Random Initialization)
- 固定值初始化
- 基于方差缩放的参数初始化
 - Xavier初始化
 - He初始化
- 正交初始化

表 7.2 Xavier 初始化和 He 初始化的具体设置情况

初始化方法	激活函数	均匀分布 $[-r, r]$	高斯分布 $\mathcal{N}(0, \sigma^2)$
Xavier 初始化	Logistic	$r = 4\sqrt{\frac{6}{M_{l-1}+M_l}}$	$\sigma^2 = 16 \times \frac{2}{M_{l-1}+M_l}$
Xavier 初始化	Tanh	$r = \sqrt{\frac{6}{M_{l-1}+M_l}}$	$\sigma^2 = \frac{2}{M_{l-1}+M_l}$
He 初始化	ReLU	$r = \sqrt{\frac{6}{M_{l-1}}}$	$\sigma^2 = \frac{2}{M_{l-1}}$

数据归一化对梯度的影响

● 尺度不变性

- 机器学习算法在缩放全部或部分特征后不影响学习和预测

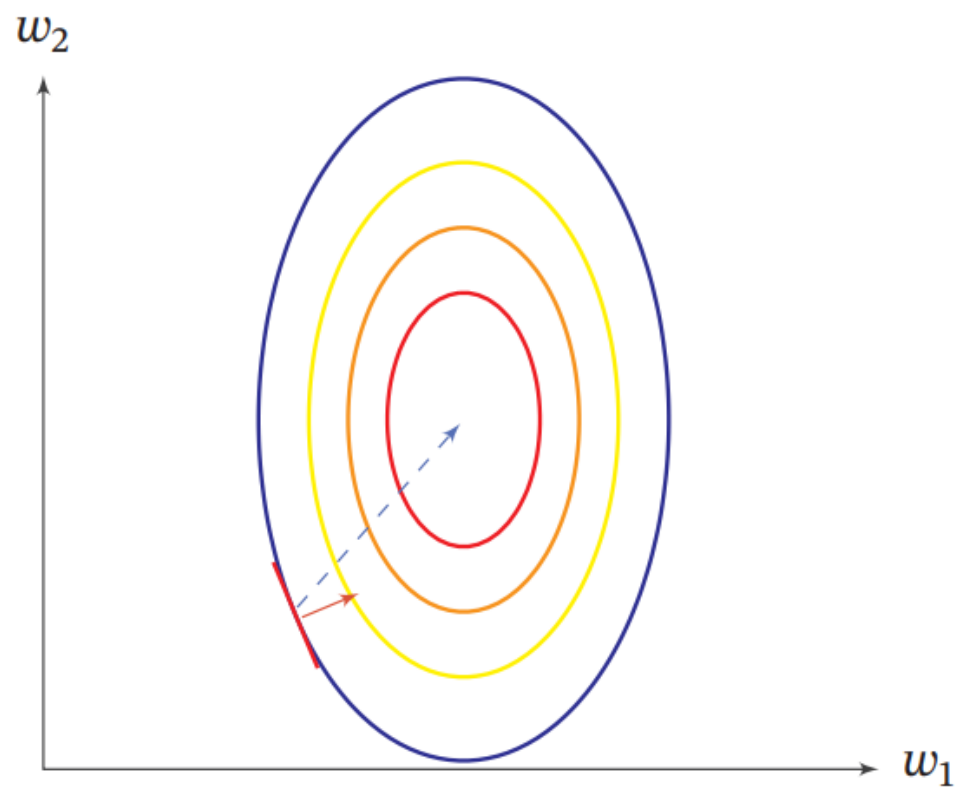
ID	长度 (cm)	宽度 (cm)	类别
1	4.2	5.9	橙子
2	6.4	7.0	橙子
3	8.3	5.5	柠檬
...			
12	5.9	8.9	橙子
13	8.8	9.3	橙子
14	10.2	6.7	柠檬



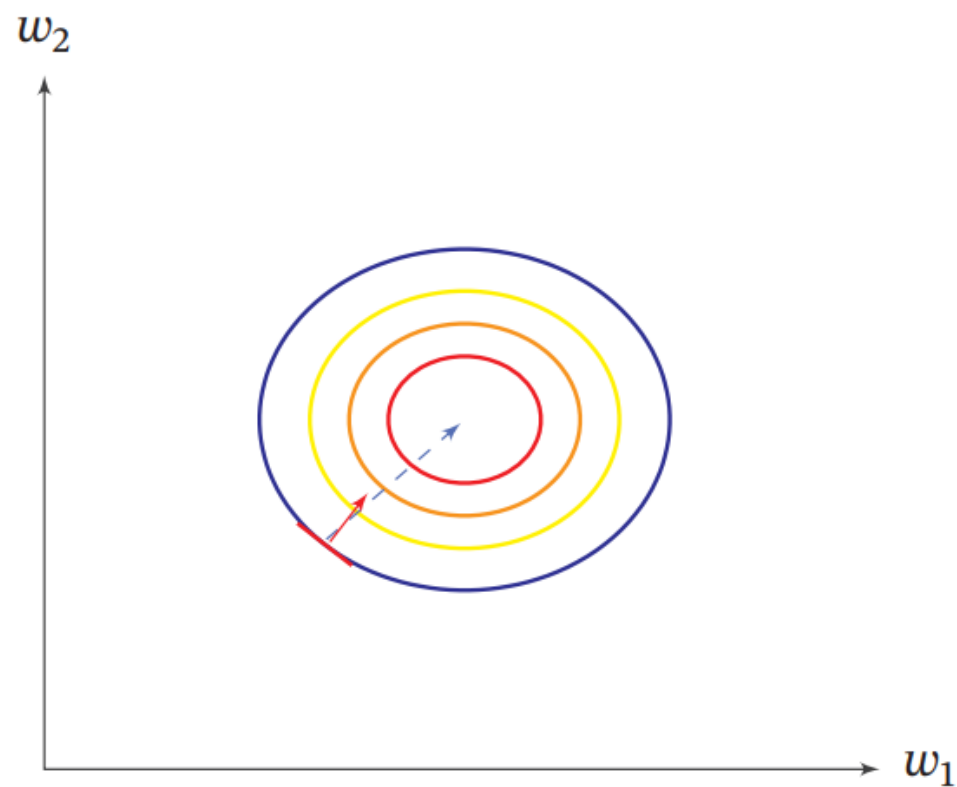
ID	长度 (cm)	宽度(mm)	类别
1	4.2	59	橙子
2	6.4	70	橙子
3	8.3	55	柠檬
...			
12	5.9	89	橙子
13	8.8	93	橙子
14	10.2	67	柠檬

不同输入特征的尺度差异过大，会导致哪些问题？

数据归一化对梯度的影响



(a) 未归一化数据的梯度

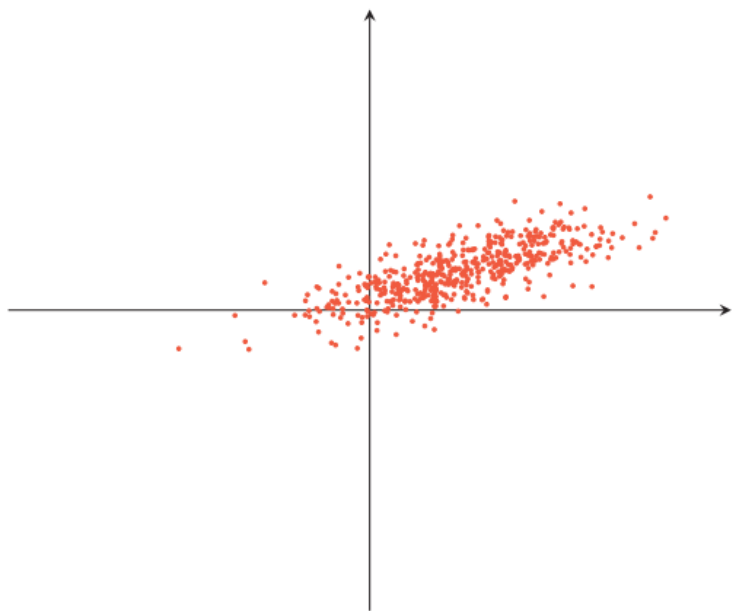


(b) 归一化数据的梯度

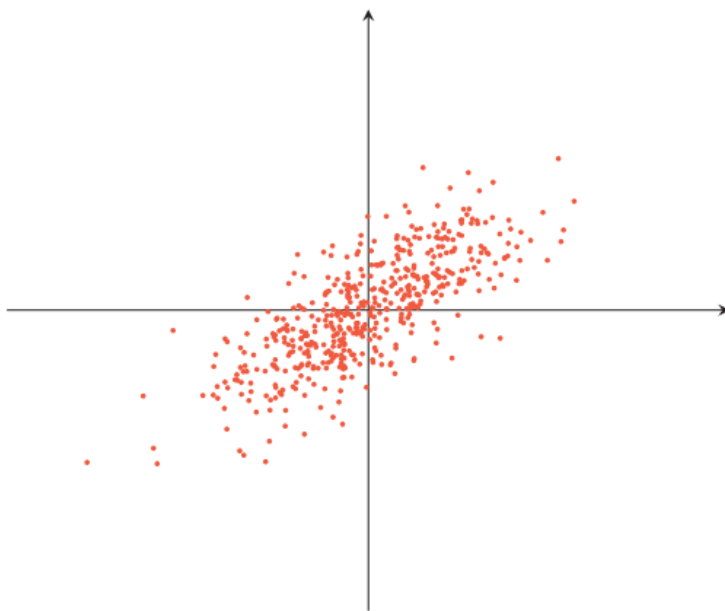
数据预处理

- 最小最大值归一化 (Min-Max Normalization) $\hat{x}^{(n)} = \frac{x^{(n)} - \min_n(x^{(n)})}{\max_n(x^{(n)}) - \min_n(x^{(n)})}$
- 标准化 (Standardization) $\hat{x}^{(n)} = \frac{x^{(n)} - \mu}{\sigma}$
- 白化 (Whitening) : 用主成分分析 (PCA) 方法

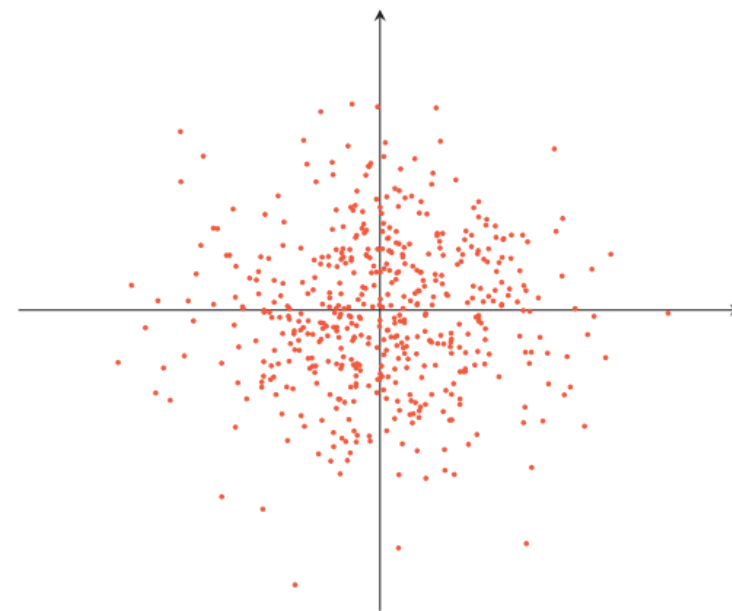
原始数据



标准归一化



PCA 白化



归一化 (Normalization)



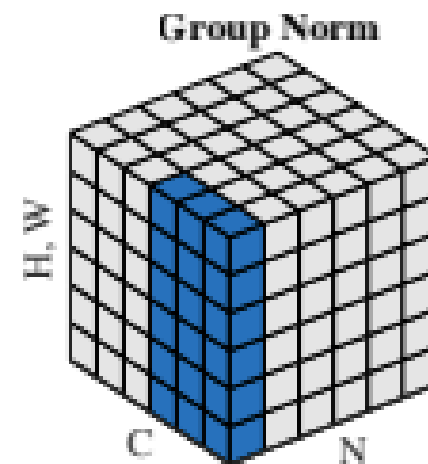
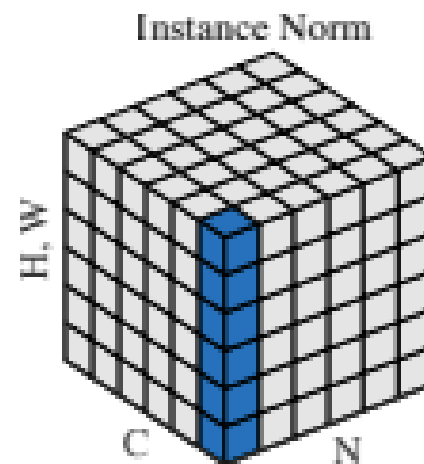
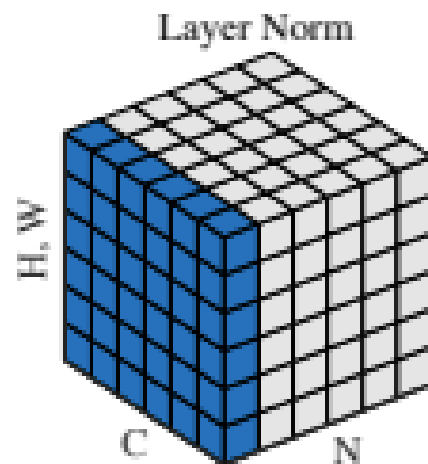
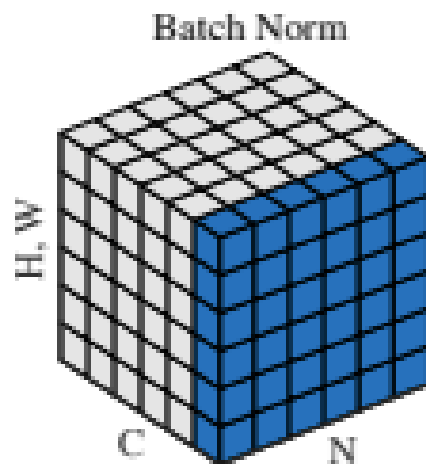
归一化

- 归一化能够带来：

- 更好的尺度不变性（缓解内部协变量偏移问题）
- 更平滑的优化地形

- 具体可分为：

- 批量归一化（Batch Normalization, BN）
- 层归一化（Layer Normalization, LN）
- 实例归一化（Instance Normalization）



Batch Norm (BN)

- 给定一个包含K个样本的小批量样本集合，计算均值和方差

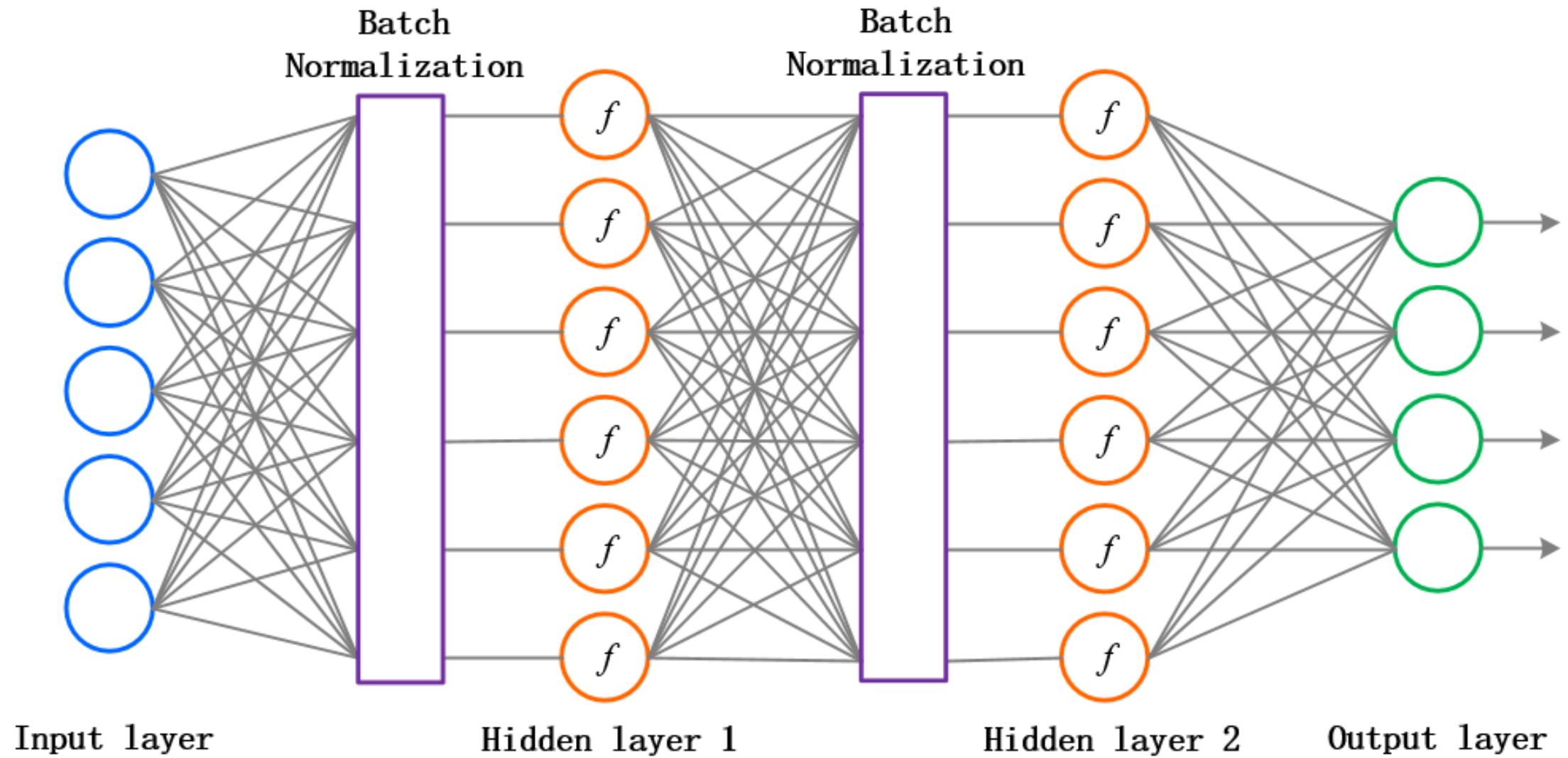
$$\mu_{\mathcal{B}} = \frac{1}{K} \sum_{k=1}^K \mathbf{z}^{(k,l)},$$
$$\sigma_{\mathcal{B}}^2 = \frac{1}{K} \sum_{k=1}^K (\mathbf{z}^{(k,l)} - \mu_{\mathcal{B}}) \odot (\mathbf{z}^{(k,l)} - \mu_{\mathcal{B}}).$$

- 批量归一化

$$\hat{\mathbf{z}}^{(l)} = \frac{\mathbf{z}^{(l)} - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}} \odot \gamma + \beta$$
$$\triangleq \text{BN}_{\gamma, \beta}(\mathbf{z}^{(l)}),$$

思考：训练阶段VS推理阶段？

Batch Norm (BN)



Batch Norm (BN)

- **优点:**

- 提高优化效率
- 是一种隐形的正则化方法：在训练时，神经网络对样本的预测不仅与自身有关，也与同一个Batch中的其他样本相关，由于选取批次时有随机性，因此使得神经网络不会“过拟合”到某个特定样本，从而提高网络的泛化能力

- **缺点:**

- 每个Batch样本数不能太少
- 循环神经网络不能使用

Layer Norm (LN)

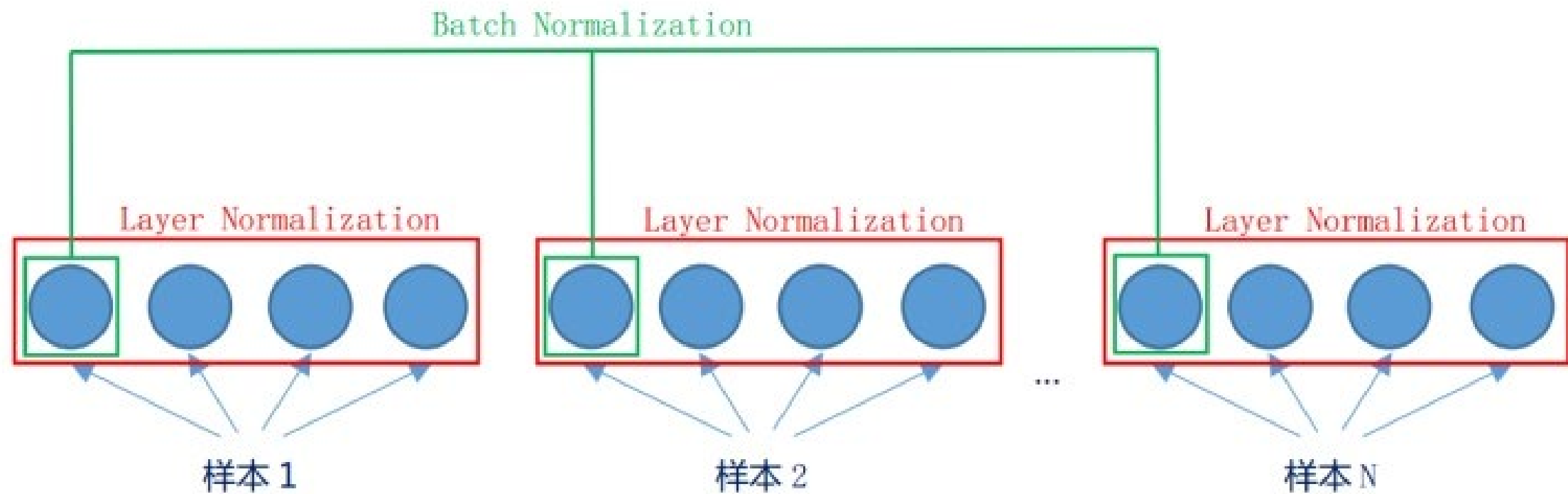
- 第 l 层神经元的净输入为 $z^{(l)}$

$$\mu^{(l)} = \frac{1}{n^l} \sum_{i=1}^{n^l} z_i^{(l)},$$
$$\sigma^{(l)2} = \frac{1}{n^l} \sum_{i=1}^{n^l} (z_i^{(l)} - \mu^{(l)})^2,$$

- 层归一化定义为

$$\hat{z}^{(l)} = \frac{z^{(l)} - \mu^{(l)}}{\sqrt{\sigma^{(l)2} + \epsilon}} \odot \gamma + \beta$$
$$\triangleq \text{LN}_{\gamma, \beta}(z^{(l)}),$$

Batch Norm VS Layer Norm



超参数优化



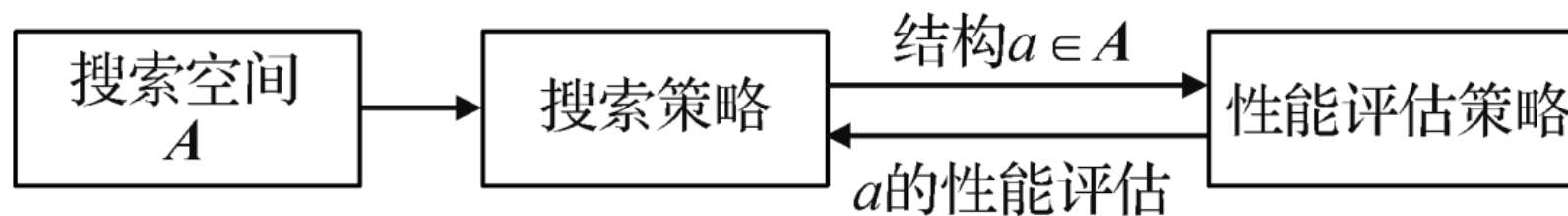
超参数优化

● 超参数

- 网络结构，包括神经元之间的连接关系、层数、每层的神经元数量、激活函数的类型等。
- 优化参数，包括优化方法、学习率、小批量的样本数量等。
- 正则化系数。

● 优化方法

- 网格搜索
- 随机搜索
- 贝叶斯优化
- 动态资源分配
- 神经架构搜索NAS



超参数优化

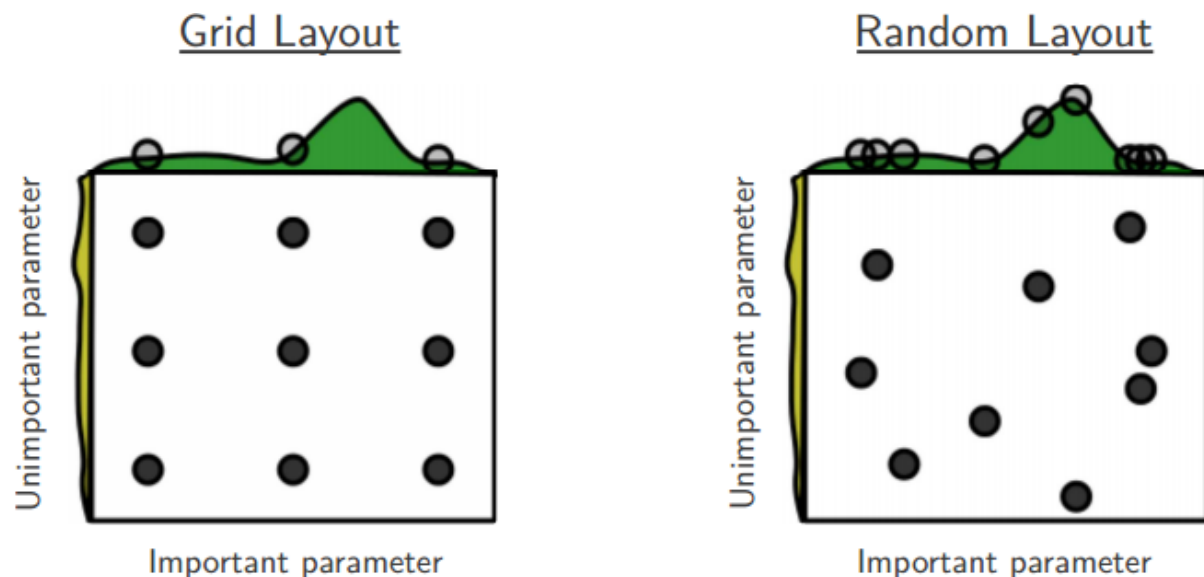
- 网格搜索 (Grid Search)

- 假设总共有K 个超参数，第k个超参数的可以取 m_k 个值。
- 如果参数是连续的，可以将参数离散化，选择几个 “经验” 值。比如学习率 α ，我们可以设置

$$\alpha \in \{0.01, 0.1, 0.5, 1.0\}$$

- 这些超参数可以有 $m_1 \times m_2 \times \cdots \times m_K$ 个取值组合。

- 随机搜索 (Random Search)



Grid and random search of nine trials for optimizing a function $f(x,y) = g(x) + h(y) \approx g(x)$ with low effective dimensionality. Above each square $g(x)$ is shown in green, and left of each square $h(y)$ is shown in yellow. With grid search, nine trials only test $g(x)$ in three distinct places. With random search, all nine trials explore distinct values of g . This failure of grid search is the rule rather than the exception in high dimensional hyper-parameter optimization.

网络正则化



正则化 (Regularization)

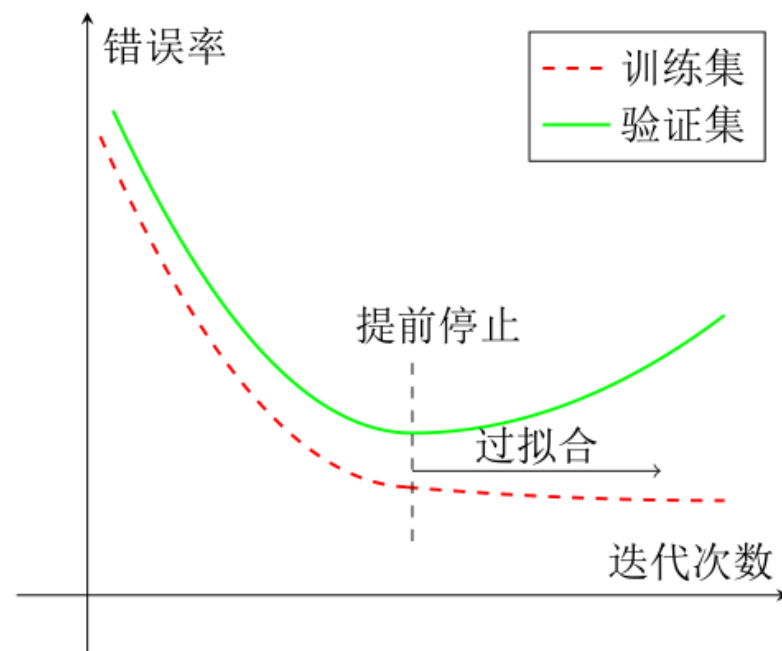
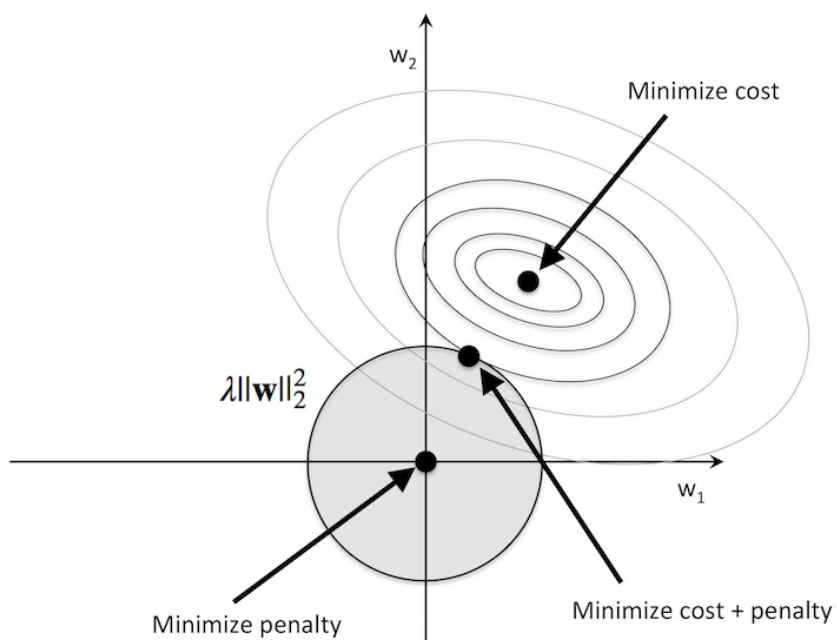
所有损害优化的方法都是正则化。

增加优化约束

干扰优化过程

L1/L2约束、数据增强

权重衰减、随机梯度下降、提前停止

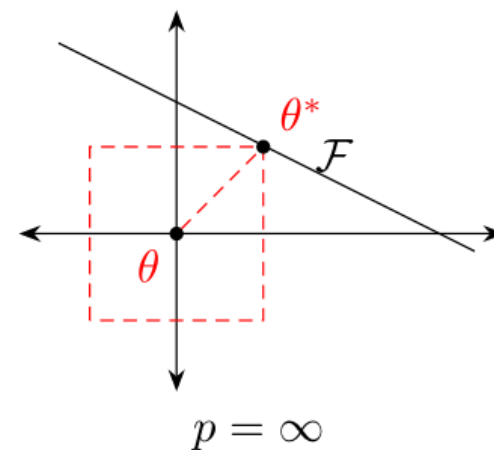
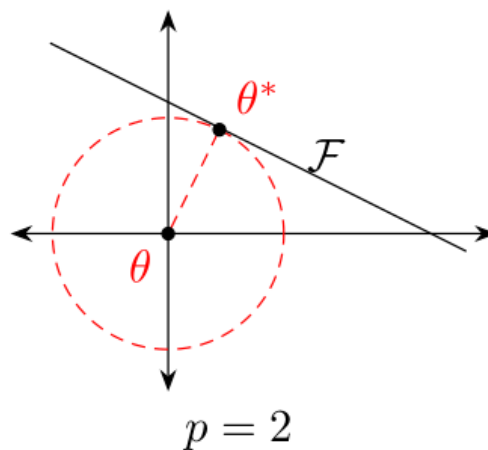
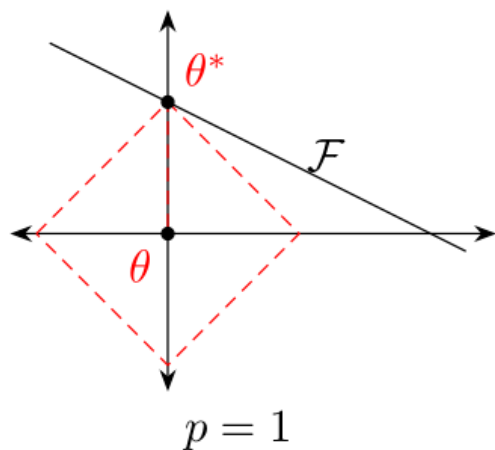


ℓ_1 和 ℓ_2 正则化

- 优化问题可以写为

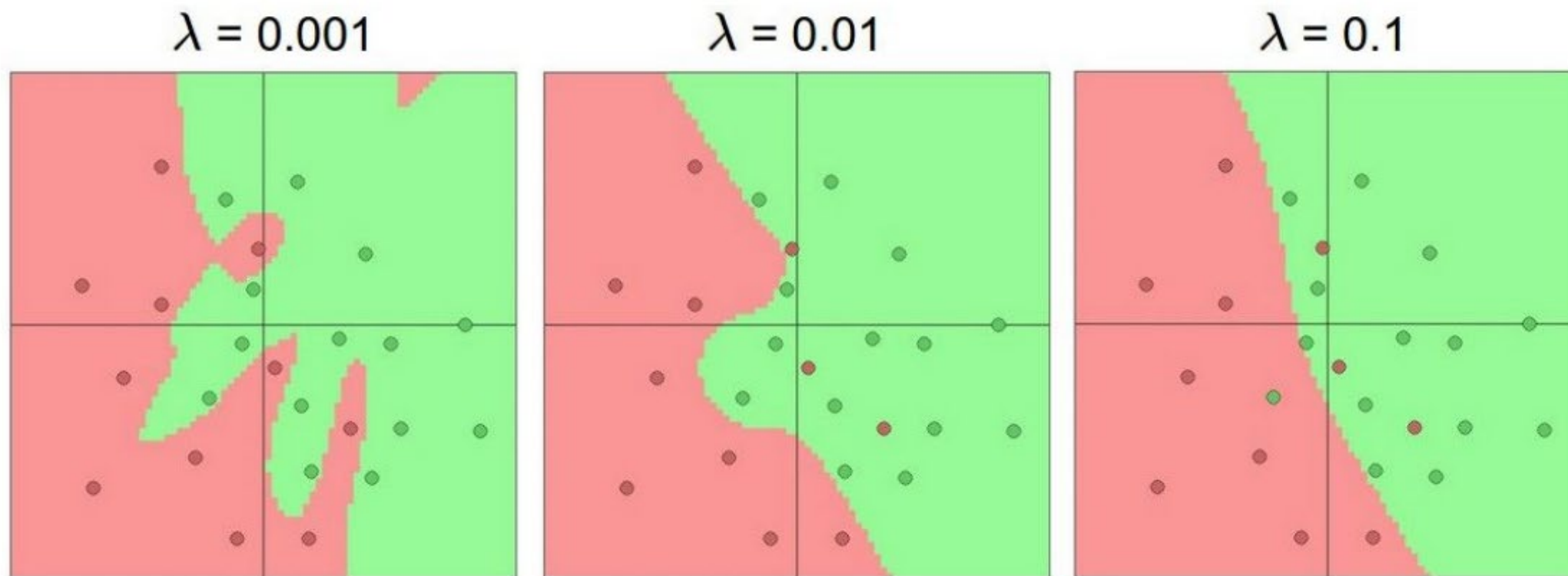
$$\theta^* = \arg \min_{\theta} \frac{1}{N} \sum_{n=1}^N \mathcal{L}(y^{(n)}, f(\mathbf{x}^{(n)}, \theta)) + \lambda \ell_p(\theta)$$

- ℓ_p 为范数函数, p 的取值通常为 $\{1, 2\}$ 代表 ℓ_1 和 ℓ_2 范数, λ 为正则化系数。



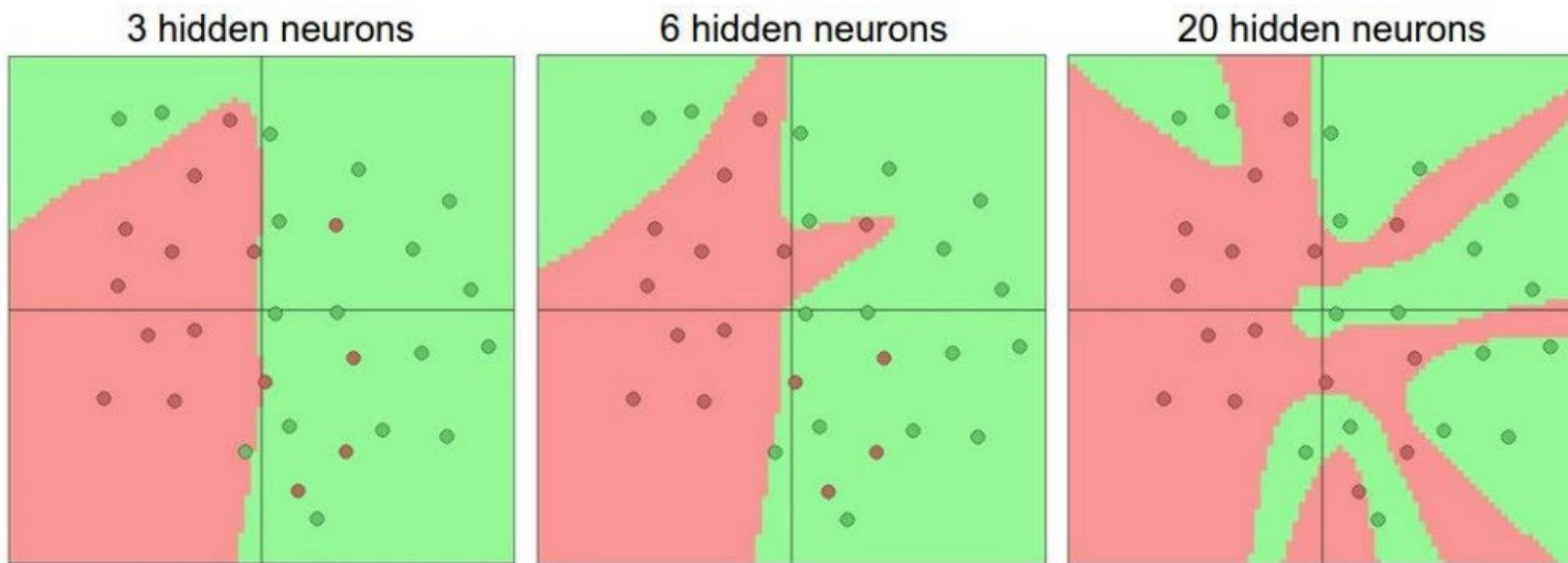
神经网络示例

- 不同的正则化系数



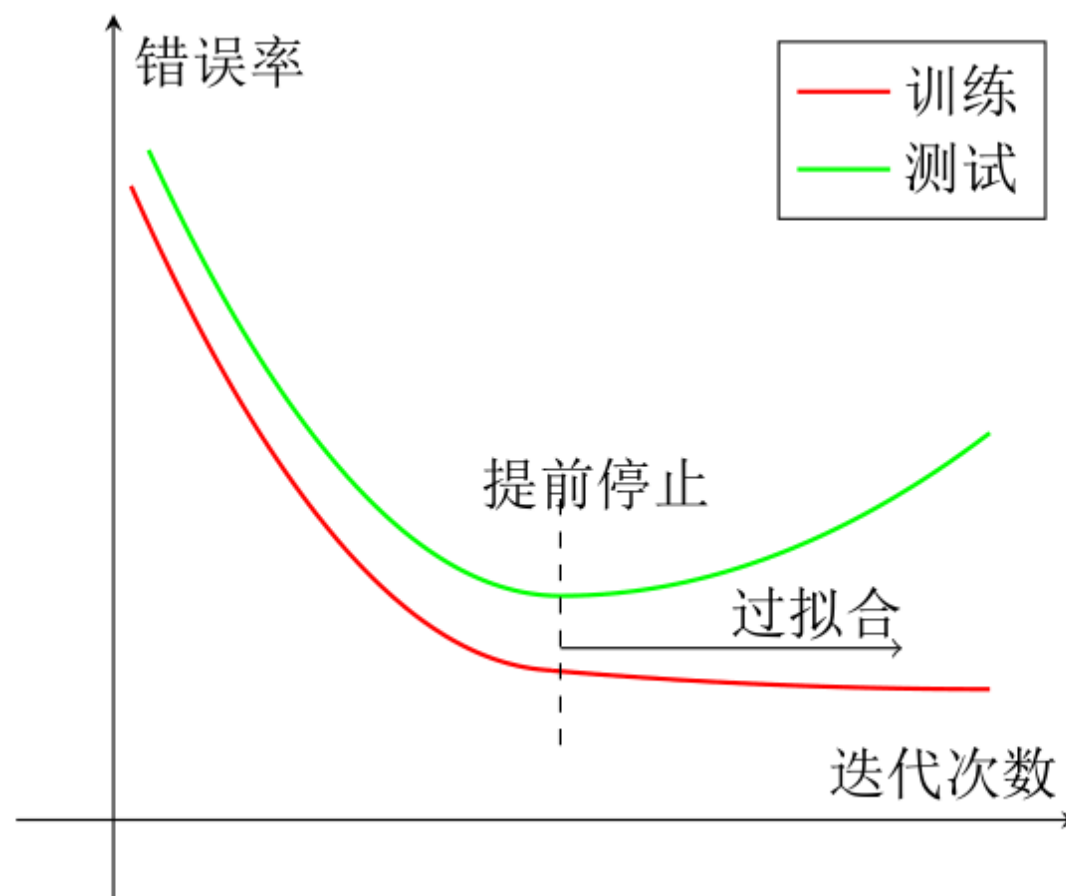
神经网络示例

- 隐藏层的不同神经元个数



提前停止 (Early-Stop)

- 使用一个验证集 (Validation Dataset) 来测试每一次迭代的参数在验证集上是否最优。如果在验证集上的错误率不再下降，就停止迭代。



权重衰减 (Weight Decay)

- 在每次参数更新时，引入一个衰减系数 w 。

$$\theta_t \leftarrow (1 - w)\theta_{t-1} - \alpha \mathbf{g}_t$$

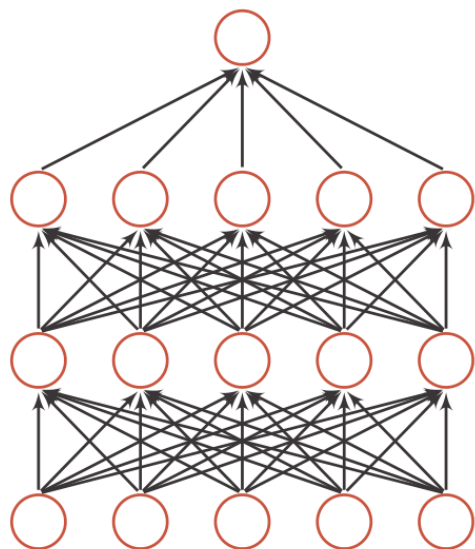
- 在标准的随机梯度下降中，权重衰减正则化和 ℓ_2 正则化的效果相同。
- 在较为复杂的优化方法（比如Adam）中，权重衰减和 ℓ_2 正则化并不等价。

丢弃法 (Dropout)

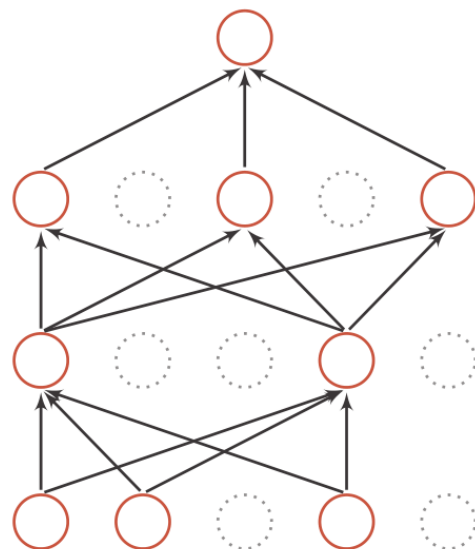
- 对于一个神经层 $y = f(Wx + b)$, 引入一个丢弃函数 $d(\cdot)$ 使得 $y = f(Wd(x) + b)$ 。

$$d(\mathbf{x}) = \begin{cases} \mathbf{m} \odot \mathbf{x} & \text{当训练阶段时} \\ p\mathbf{x} & \text{当测试阶段时} \end{cases}$$

- 其中 $m \in \{0,1\}^d$ 是丢弃掩码 (dropout mask), 通过以概率为 p 的伯努利分布随机生成。



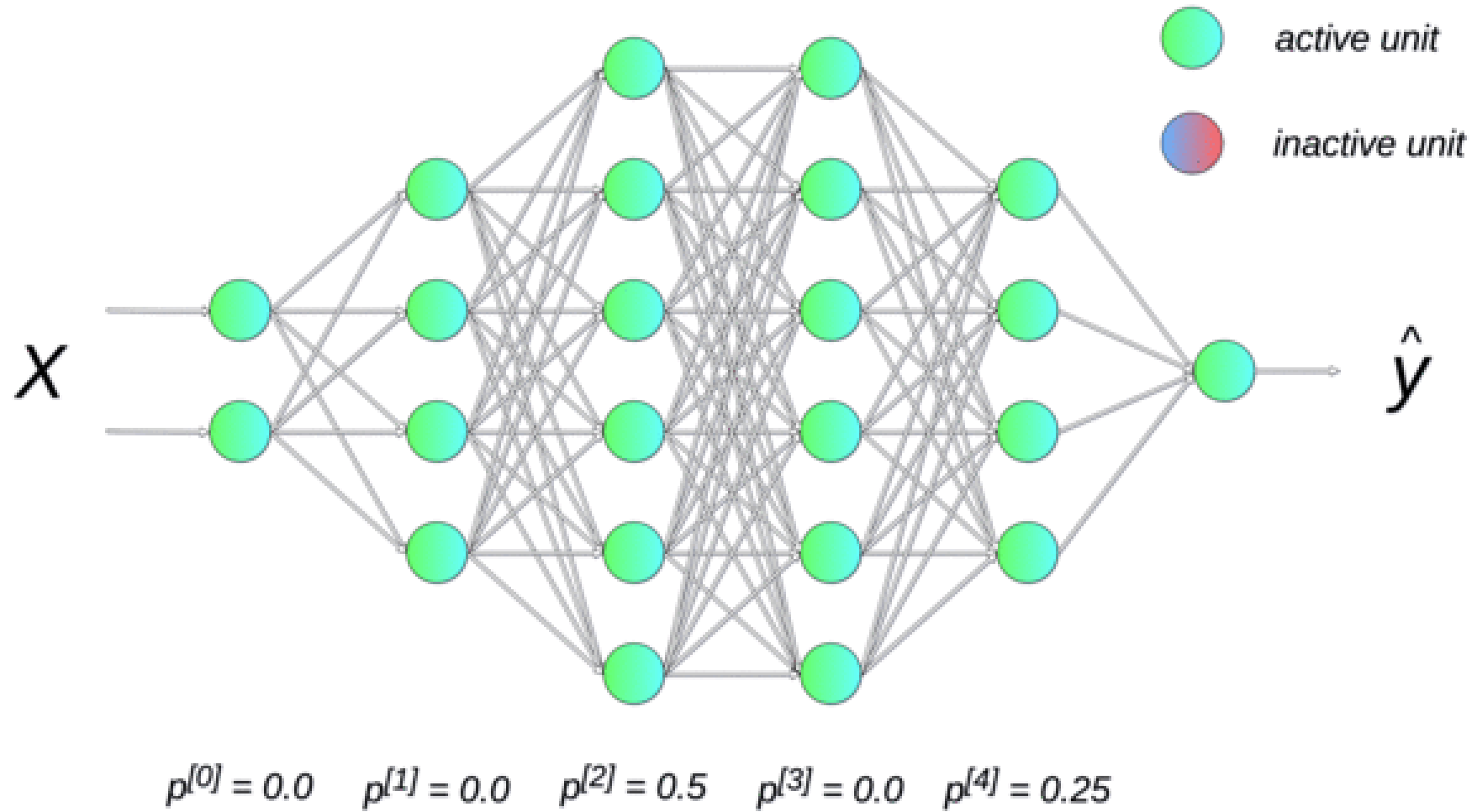
(a) 标准网络



(b) Dropout 后的网络

思考：训练阶段VS推理阶段？

丢弃法 (Dropout)



- 集成学习的解释

- 每做一次丢弃，相当于从原始的网络中采样得到一个子网络。如果一个神经网络有 n 个神经元，那么总共可以采样出 2^n 个子网络。

- 贝叶斯学习的解释

$$\begin{aligned}\mathbb{E}_{q(\theta)}[y] &= \int_q f(\mathbf{x}, \theta) q(\theta) d\theta \\ &\approx \frac{1}{M} \sum_{m=1}^M f(\mathbf{x}, \theta_m),\end{aligned}$$

- 其中 $f(x, \theta_m)$ 为第 m 次应用丢弃方法后的网络。

总结



总结

● 模型

- 用 ReLU 作为激活函数
- 分类时用交叉熵作为损失函数
- 逐层归一化

● 优化

- SGD+mini-batch (动态学习率、Adam算法优先)
- 每次迭代都重新随机排序
- 数据预处理 (标准归一化)
- 参数初始化

● 正则化

- ℓ_1 和 ℓ_2 正则化 (跳过前几轮)
- Dropout
- Early-stop
- 数据增强

Q&A